

A Practical Survey on Faster and Lighter Transformers

QUENTIN FOURNIER, Polytechnique Montréal, Canada

GAËTAN MARCEAU CARON, Mila - Quebec AI Institute, Canada

DANIEL ALOISE, Polytechnique Montréal, Canada

Recurrent neural networks are effective models to process sequences. However, they are unable to learn long-term dependencies because of their inherent sequential nature. As a solution, Vaswani et al. introduced the Transformer, a model solely based on the attention mechanism that is able to relate any two positions of the input sequence, hence modelling arbitrary long dependencies. The Transformer has improved the state-of-the-art across numerous sequence modelling tasks. However, its effectiveness comes at the expense of a quadratic computational and memory complexity with respect to the sequence length, hindering its adoption. Fortunately, the deep learning community has always been interested in improving the models' efficiency, leading to a plethora of solutions such as parameter sharing, pruning, mixed-precision, and knowledge distillation. Recently, researchers have directly addressed the Transformer's limitation by designing lower-complexity alternatives such as the Longformer, Reformer, Linformer, and Performer. However, due to the wide range of solutions, it has become challenging for researchers and practitioners to determine which methods to apply in practice to meet the desired tradeoff between capacity, computation, and memory. This survey addresses this issue by investigating popular approaches to make Transformers faster and lighter and by providing a comprehensive explanation of the methods' strengths, limitations, and underlying assumptions.

CCS Concepts: • **Computing methodologies** → **Neural networks**;

Additional Key Words and Phrases: Deep learning, efficient transformer, self-attention, survey

ACM Reference format:

Quentin Fournier, Gaétan Marceau Caron, and Daniel Aloise. 2023. A Practical Survey on Faster and Lighter Transformers. *ACM Comput. Surv.* 55, 14s, Article 304 (July 2023), 40 pages.

<https://doi.org/10.1145/3586074>

1 INTRODUCTION

Sequences arise naturally in a wide range of domains, notably in natural language, biology, and software executions. Rumelhart et al. [97] introduced a family of models called **recurrent neural networks (RNNs)** based on the idea of parameter sharing to process variable-length sequences. Given an input sequence X comprising n tokens $\mathbf{x}^{(i)}$ of dimension d , recurrent neural networks iteratively construct a sequence of hidden representations $\mathbf{h}^{(i)}$ and produce a sequence of

We would like to gratefully acknowledge the Natural Sciences and Engineering Research Council of Canada (NSERC), Prompt, Ericsson, Ciena, and EfficIOS for funding this research.

Authors' addresses: Q. Fournier and D. Aloise, Polytechnique Montréal, 2500 Chemin de Polytechnique, Montréal, Quebec, Canada, H3T 1J4; emails: {quentin.fournier, daniel.aloise}@polymtl.ca; G. M. Caron, Mila - Quebec AI Institute, 6666 Rue Saint-Urbain, Montréal, Quebec, Canada, H2S 3H1; email: gaetan.marceau.caron@mila.quebec.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0360-0300/2023/07-ART304 \$15.00

<https://doi.org/10.1145/3586074>

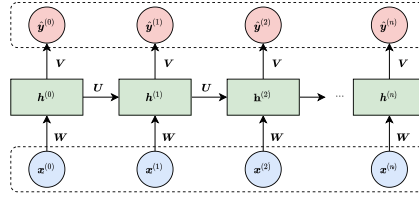


Fig. 1. The computational graph of a recurrent neural network. The input and output sequences are depicted in blue and red, respectively. The position, also known as the timestep, is indicated in superscript. The weight matrices W , U , and V are shared across all positions. Reproduced with permission [31]. Copyright 2021 IEEE.

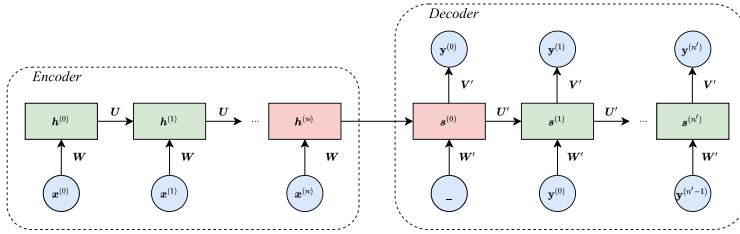


Fig. 2. The sequence-to-sequence framework where the encoder and decoder are recurrent neural networks. The input sequence (blue) is encoded into a fixed-size context $h^{(n)}$ (red), which serves as the initial state of the decoder.

outputs $y^{(i)}$, as illustrated in Figure 1. Unfortunately, vanilla RNNs often suffer from vanishing or exploding gradients, which prevent them from learning long-term dependencies. Hochreiter and Schmidhuber [44] addressed this limitation with the now widely popular **long short-term memory (LSTM)** network, which circumvents the gradient issues with paths through time. Cho et al. [17] later improved over the LSTM with the simpler **gated recurrent unit (GRU)**.

Recurrent neural networks align the input and output sequences, that is, there is a one-to-one mapping between the two sequences. Depending on the task, this property of RNNs may be too restrictive: For instance, translation requires outputting a sequence whose size is often different from that of the input while aligning tokens at different positions. Sutskever et al. [112] addressed this limitation by introducing the sequence-to-sequence framework in which a first network (encoder) processes the entire input sequence and returns its last hidden representation $h^{(n)}$, effectively encoding the input into a fixed-size vector called context. The context then serves as the initial state for a second network (decoder), which generates the output sequence in an autoregressive manner. The decoding stops when a special end-of-sequence token is generated. Figure 2 illustrates the sequence-to-sequence framework.

In practice, the fixed-size nature of the hidden representation hinders the effectiveness of recurrent neural networks [15]. Indeed, as the input sequence is processed, information is iteratively stored into the hidden representation that may be too small to retain all the relevant information for the task. In that case, useful data is inevitably lost, which may significantly impact the model's performance. Bahdanau et al. [3] introduced an alignment mechanism called inter-attention to overcome the bottleneck of the sequence-to-sequence framework. This attention mechanism computes a different representation of the input for each output step, effectively allowing the decoder to “look at” the relevant part(s) of the input for each output step. Thereby, the inter-attention alleviates the encoder's burden to encode all information about the input sequence into a fixed-size vector. Formally, the context is the weighted sum of the encoder's hidden representations h_i , for $i = 1, \dots, n$, where the weights are computed with a feed-forward neural network. For a

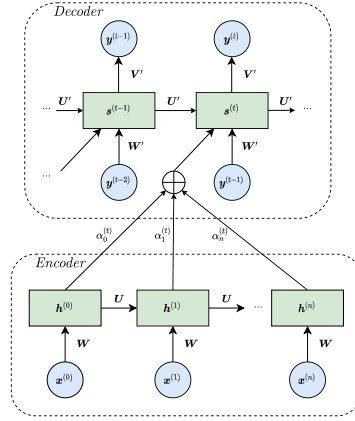


Fig. 3. The inter-attention mechanism. The attention weight $\alpha_i^{(t)}$ depicts the strength with which the i th encoder hidden representation $h^{(i)}$ contributes to the context of t th decoder step. Reproduced with permission [31]. Copyright 2021 IEEE.

comprehensive survey of the attention mechanism, we refer the reader to Galassi et al. [33] and Weng [130]. Figure 3 illustrates the inter-attention mechanism.

Moreover, recurrent neural networks do not scale efficiently to longer sequences due to their iterative nature [121]. In particular, RNNs struggle to learn dependencies between distant positions. One measure of this limitation is the **relative effective context length (RECL)** introduced by Dai et al. [22]. The RECL is the largest context length that leads to a substantial relative gain over the best model. In other words, increasing the context length over the RECL yields a negligible increase in performance over the best model. The authors estimated that the relative effective context length of LSTMs on natural language data is limited to approximately 400 words. Besides, Khandelwal et al. [58] empirically observed that LSTMs sharply model recent positions but only vaguely remember the distant past.

1.1 Transformer

This inherent limitation of recurrent neural networks has prevented them from being successfully applied to domains that require processing long sequences such as DNA. To overcome this limitation, Vaswani et al. [121] introduced the *Transformer*, a sequence-to-sequence model built without recurrences. Instead, the Transformer relies solely on the attention mechanism: the inter-attention between the encoder and decoder (see Figure 3), and the self-attention, also known as intra-attention, within the encoder and decoder. The self-attention's main advantage is its ability to relate any two positions of the input sequence regardless of their distance, thus increasing performance significantly on a wide range of tasks, including **natural language processing (NLP)** [10, 24, 121], computer vision [12, 27, 57], speech recognition [40, 110, 140], and biological sequence analysis [139]. Karita et al. [55] evaluated a Transformer against a sequence-to-sequence Bi-LSTM baseline on **automatic speech recognition (ASR)**, **speech translation (ST)**, and **text-to-speech (TTS)**. The attention-based models outperformed the baseline on 13 corpora out of 15 for monolingual ASR and realized more than 10% relative improvement in 8 languages out of 10 for multilingual ASR. The Transformer improved the BLEU score from 16.5 for the baseline to 17.2 on ST while performing on par for TTS. Table 1 reports the performance improvements brought by popular Transformer architectures over previous state-of-the-art models across different domains. As of this article's writing, the Transformer has become the de facto model for numerous sequence processing tasks.

Table 1. Relative Improvements Brought by Popular Transformer Architectures over Previous State-of-the-art Models

Task	Dataset	Previous SOTA	Transformer's Architecture	Relative Improvement
Machine Translation	newstest2014 (EN-to-DE)	MoE (GNMT) [103]	Vanilla [121]	9.1% (+2.37 BLEU ²)
	newstest2014 (EN-to-FR)			3.1% (+1.24 BLEU)
Image Classification	ImageNet	Noisy Student (EfficientNet-L2) [134]	ViT [27]	0.2% (+0.15% Acc)
	CIFAR-10	BiT-L (ResNet152x4) [60]		0.1% (+0.13% Acc)
	CIFAR-100			1.1% (+1.04% Acc)
	VTAB (19 tasks)			1.8% (+1.34% Acc)
Text Classification	SST2	Sparse byte mLSTM [39]	BERT [24]	1.8% (+1.70% Acc)
	CoLA	Single-task BiLSTM + ELMo + Attn [124]		72.9% (+25.5 MC ³)
Speech-to-text	librispeech (test-clean)	LAS (LSTM) [13, 86]	Convformer [40]	13.6% (−0.3 WER ⁴)
	librispeech (test-other)			25.0% (−1.3 WER)

Absolute differences are reported between parentheses. Sources are: [121] for machine translation, [27] for image classification, [24, 91] for text classification, and [69] for speech-to-text.

As an illustration of an end-to-end application of the Transformer, let us consider the speech recognition task. In hybrid approaches, the recognition system consists of independently trained machine learning components, often an acoustic model, a pronunciation model, and a language model. Instead, in end-to-end approaches, the recognition system consists of a single model comprising several parts trained together. Zhang et al. [140] introduced an end-to-end speech recognition model based on Transformer encoders called the Transformer Transducer that outperformed previous hybrid and end-to-end approaches on the LibriSpeech benchmarks.

The Transformer's capacity comes at the cost of a quadratic computational and memory complexity with respect to the sequence length. Therefore, training large Transformers is prohibitively slow and expensive. For instance, Liu et al. [74] introduced RoBERTa, which was pre-trained on 1,024 high-end V100 **graphics processing units (GPUs)** for approximately a day. Although numerous large pre-trained Transformers have been publicly released, fine-tuning them on the tasks of interest is still computationally expensive. Furthermore, the sequence lengths are restricted by the amount of memory available. Indeed, practitioners typically use large mini-batches with relatively short sequences, because the Transformer's optimization is known to be particularly unstable with small mini-batches. Typically, a GPU with 16 GB of memory handles sequences up to 512 words. Consequently, there exists an actual need for lighter and faster Transformers, as only a few large organizations can afford to train massive models. As of the writing of this article, the largest dense Transformer is GPT-3 [10], which requires 355 years to train on a V100 GPU, costing around \$4,600,000 of cloud instances.¹

1.2 Lighter and Faster Transformers

Over the years, numerous approaches have been proposed to reduce the computational and memory costs of neural networks, many of which have been applied to Transformers. In this article, such methods are referred to as *general*, since they apply, and have been applied, to a wide range of models. General methods are often orthogonal, and consequently, several of them may be

¹<https://lambdalabs.com/blog/demystifying-gpt-3>.

²**Bilingual evaluation understudy (BLEU)**, higher is better.

³**Matthews correlation (MC)** coefficient, higher is better.

⁴**Word error rate (WER)**, lower is better.

combined to precisely fine-tune the network's capacity, computational cost, and memory usage. However, general methods may be insufficient, as the model complexity typically remains unchanged. Therefore, many works introduced lower-complexity variations of the Transformer, referred to as x-formers. In this survey, the Transformer's alternatives are categorized depending on whether they sparsify the attention, factorize it, or modify the network's architecture. Please note that this survey aims to provide a comprehensive summary of the methods that improve the Transformer's efficiency and that fine-grained taxonomies have already been proposed by Tay et al. [116] and Lin et al. [68]. Accordingly, our taxonomy will remain purposefully coarse.

Recently, Tolstikhin et al. [118] and Liu et al. [70], among others, argued that the powerful yet expensive self-attention mechanism is not necessary to achieve state-of-the-art results and thus challenged the preconception that the self-attention is the source of the Transformer's success. Consequently, they introduced networks without self-attention that are competitive with Transformers for image classification and language modelling at the same computational cost. Yu et al. [137] expanded on this idea with a more general and flexible architecture called MetaFormer, where the mechanism to relate the tokens is not specified, while the other components are kept the same as the Transformer. Despite the recent success of attention-free architectures, such networks are outside the scope of this article, as they arguably remove the Transformer's core mechanism and are discussed in the appendices.

The remainder of this survey is organized as follows: Section 2 introduces the Transformer's architecture and the origin of the quadratic complexity. Section 3 investigates the popular general methods that have been applied to Transformers to reduce the computations and memory footprint. Section 4 explores the recent lower-complexity Transformers. Section 5 explains the limitations of the different approaches and the current evaluation methodology, Section 6 provides a discussion on the broader impact of lighter and faster Transformers, and Section 7 points out potential future research directions. Finally, Section 8 concludes this survey. Practitioners and researchers can find detailed practical guidelines regarding the general and specialized methods in the appendices, as well as a summary of the specialized methods (see Table 4) and a discussion about some of the most popular attention-free alternatives.

2 TRANSFORMER

This section formally introduces the attention mechanism, the Transformer's architecture, and the root cause of its quadratic complexity.

2.1 Attention Mechanism

The attention mechanism relies on three matrices, namely, $Q, K, V \in \mathbb{R}^{n \times d}$, commonly referred to as "queries," "keys," and "values," respectively. The attention outputs the sum of the values weighted by a compatibility or alignment score between each token, which is computed with the function $\text{Score}(Q, K) \in \mathbb{R}^{n \times n}$. Intuitively, if the i th query is highly compatible with the j th key, then the j th value greatly contributes to the i th attention's output. The attention mechanism may be written as:

$$\text{Attention}(Q, K, V) = \text{Score}(Q, K)V. \quad (1)$$

Since the compatibility score directly controls the alignment between the tokens, many functions have been proposed. In the original paper, the Transformer relies on the *scaled dot product attention*. The *dot product* refers to the computation of the compatibility score between a single query and a single key. In practice, however, the compatibility scores are computed simultaneously for every query and key by multiplying Q with K^T . Indeed, the (i, j) entry of the QK^T multiplication is equal to the dot product between the i th query and the j th key. To obtain a

probability distribution over the positions, referred to as attention weights, each row of $\mathbf{QK}^\top$ is passed through a Softmax function defined as follows:

$$\text{Softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad \text{for } i = 1, \dots, n, \quad (2)$$

where $\mathbf{x} \in \mathbb{R}^n$. Since the dot product grows large in magnitude for large values of d , thereby pushing the Softmax into a region of small gradients, a *scaling* factor $\sqrt{d}$ is introduced. Thus, the scaled dot product attention is given by:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right) \mathbf{V}. \quad (3)$$

Nonetheless, the attention presented above may not be flexible enough if the relevant information for the task is scattered across different regions of the input space. That is due in part to the Softmax being exponential, which amplifies the differences between the values. As a result, only a few attention weights are large, i.e., only a few positions are strongly attended. Vaswani et al. [121] addressed this limitation with the multi-head attention. The d -dimensional queries, keys, and values matrices are first linearly projected h times with distinct, learned projections to d_k , d_k , and d_v dimensions, respectively. On each projection, an independent attention instance called *head* is applied, and the output of each attention head is concatenated before being linearly projected. The Transformer's multi-head scaled dot product attention is given by:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1; \dots; \text{head}_h] \mathbf{W}^O, \quad (4)$$

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{QW}_i^Q (\mathbf{KW}_i^K)^\top}{\sqrt{d_k}}\right) \mathbf{VW}_i^V, \quad (5)$$

where $\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$ are the matrices that project the queries, keys, and values into the i th subspace, respectively, and where $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$ is the matrix that computes a linear transformation of the heads. Typically, $d_k = d/h$ where d is the input and output dimension, and h is the number of heads. For the sake of clarity, methods that modify the attention will be explained in the context of a single head (see Equation (3)).

Thus far, the attention mechanism has been described as a general method. The Transformer relies on two specific instances of this mechanism: the intra-attention, popularly known as self-attention, and the inter-attention, sometimes referred to as cross-attention. In the case of inter-attention, the queries correspond to the decoder's hidden representations, and the keys and values are the encoder's outputs. It allows the decoder to look at the relevant parts of the input to produce the output. In the case of self-attention, the three matrices are linear projections of the layer's input, which allows the encoder and decoder to focus on the relevant part of the sequence for each position, similarly to the inter-attention depicted in Figure 3.

2.2 Encoder

The Transformer's encoder is a function defined as the composition of L identical layers or blocks, each composed of two sub-layers. The first sub-layer is the aforementioned self-attention mechanism. The second sub-layer is a simple fully connected feed-forward network applied position-wise, that is, independently and identically to every position. The feed-forward network increases the encoder's expressiveness and transforms the self-attention's output for the next layer.

Inspired by ResNet [42], a skip connection, shortcut connection, or residual connection is applied around each sub-layer to create a direct path for the gradient to flow throughout the network. Notably, residual connections make the training of very deep neural networks more stable.

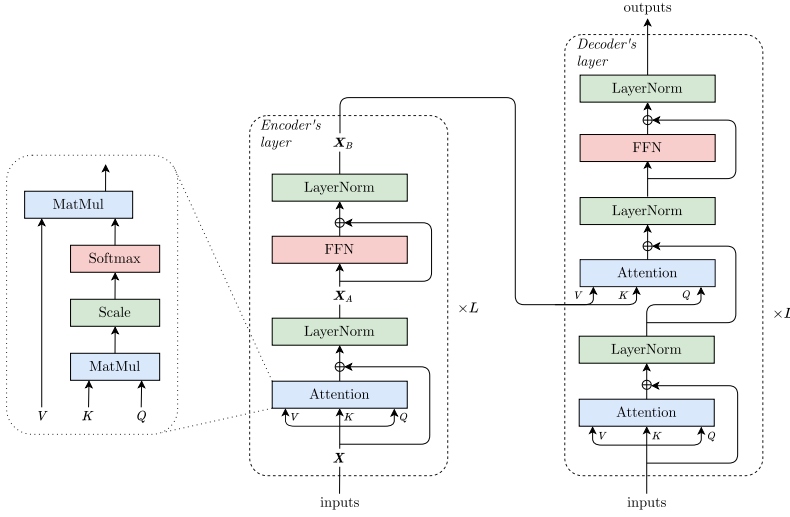


Fig. 4. The Transformer's computational graph [121]. From left to right, the scaled dot product self-attention, the encoder, and the decoder. Note that both the encoder and decoder comprise L identical layers, of which only one is depicted.

Additionally, both sub-layers' outputs are normalized after the residual connection with the layer normalization technique, referred to as LayerNorm [64]. Normalization is a widely adopted technique in deep learning that enables faster and more stable training. Although the rationale behind the normalization's empirical success is not yet fully understood [67], it has been conjectured that this results from a smoother optimization landscape, and to a lesser extent, from a reduction in internal covariance shift [100]. Figure 4 depicts the computational graph of an encoder's layer.

In natural language processing, the input sequence X would typically represent a sentence or a paragraph, and the token $\mathbf{x}^{(i)}$ would be its i th word or subword embedding. Each encoder's layer is given by:

$$X_A = \text{LayerNorm}(\text{Attention}(Q, K, V) + X), \quad (6)$$

$$X_B = \text{LayerNorm}(\text{FFN}(X_A) + X_A), \quad (7)$$

where X and X_B are the layer's input and output, respectively, and Q , K , and V are linear projections of X .

The feed-forward network is given by:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (8)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d_f}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_f \times d}$, and where d_f is the dimension of the hidden layer. Note that the feed-forward network is defined for a row vector, since it is applied position-wise, that is, it is independently and identically applied to every position or row.

Finally, the position-wise layer normalization is given by:

$$\text{LayerNorm}(\mathbf{x}) = \mathbf{g} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \mathbf{b}, \quad (9)$$

where $\odot$ denotes the element-wise (Hadamard) product, where the average μ and the standard deviation σ are computed from all of the summed inputs, where the gain $\mathbf{g}$ and the bias $\mathbf{b}$ are learned parameters of dimension d , and where ϵ is a small constant used in practice for numerical stability.

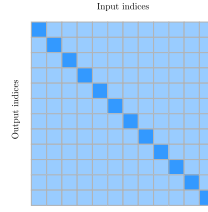


Fig. 5. The connectivity matrix of the full attention. The i th output position attends to the j th input position if, and only if, the cell (i, j) is colored. The diagonal is highlighted to ease the reading.

2.3 Decoder

The decoder is also composed of L identical layers. Although it is common for the decoder to have the same number of layers as the encoder, one may adjust their depth independently. Each decoder's layer comprises three sub-layers. The first sub-layer is the self-attention mechanism, as in the encoder, except that future positions are masked. Indeed, while the encoder is allowed to look at future positions, since the input sequence is entirely available, the decoder is autoregressive and thus cannot look at future positions, since they have not yet been predicted. Therefore, the i th position may only attend to positions less than i . The second sub-layer is the inter-attention mechanism, which helps the decoder focus on the relevant parts of the input. Finally, the third sub-layer is a simple feed-forward network. As for the encoder, a residual connection and a layer normalization are applied to each sub-layer.

Note that the decoder may be safely omitted when the task does not require the sequence-to-sequence framework, such as sentiment analysis, which predicts whether a sentence is positive. One of the most popular encoder-only Transformers is the **Bidirectional Encoder Representations from Transformers (BERT)** [24], a state-of-the-art language model that learns contextualized embeddings. Nonetheless, autoregressive tasks such as machine translation still require the sequence-to-sequence framework.

2.4 Complexity

Intuitively, the quadratic complexity emerges from the computation of the compatibility score between every pair of positions. More precisely, the QK^T multiplication requires n^2 computations and memory. Such attention is said to be full, since any output position is able to attend to any input position. The attention pattern is visualized by means of a connectivity matrix, which indicates the input positions that each output position is able to attend (see Figure 5).

What justifies such efforts from the community to improve the Transformer's efficiency? In our opinion, there are three primary motivations: affordability, scalability, and ecology.

The foremost reason is affordability. The Transformer has largely surpassed convolutional and recurrent neural networks and achieved new state-of-the-art results across many tasks. However, those networks have a linear complexity with respect to the sequence length [121], making them affordable to most researchers and practitioners. As explained by Strubell et al. [109], this creates three major issues: (1) it stifles creativity, as researchers and practitioners that do not have access to considerable resources are not able to experiment with Transformers, (2) it reinforces the "rich get richer" cycle, where successful labs and companies receive more funding due to their existing accomplishments with Transformers, and (3) it forces smaller labs and companies to rely on private cloud services that end up more expensive.

The second reason is scalability. The quadratic complexity prevents researchers and practitioners, even those with access to considerable resources, from applying Transformers on long sequences such as entire chapters or books, high-resolution images or videos, and DNA.

The third reason is ecology. It is now more apparent than ever that we must cut **carbon dioxide** (CO_2) emissions in half over the next decade to limit global warming. The large-scale infrastructures used by the deep learning community consume a considerable amount of electricity, which is mainly produced by non-renewable sources such as coal or gas [49].

Thereby, the following sections investigate popular and novel methods to make Transformers faster and lighter.

3 GENERAL APPROACHES

Computational resources have always been a limiting factor for deep learning models [63]. Therefore, numerous approaches have been proposed throughout the years to design faster and lighter models. This section introduces the most popular techniques that apply to virtually all neural networks.

Gradient Checkpointing [14]: Intermediate results computed during the forward pass, also referred to as activations, are required to compute the gradients during the backward pass; therefore, they are stored in memory. Activations typically account for most of the memory during training: Given an l -layer network, the number of intermediate results is proportional to the number of layers ($O(l)$). With gradient checkpointing, also known as rematerialization, activations are stored only for a subset of the layers. However, they must be recomputed during the backward pass, trading memory for computations. In the extreme case where no activations are stored, the memory usage becomes constant ($O(1)$) at the cost of a quadratic number of computations with respect to the number of layers ($O(l^2)$). Chen et al. [14] designed a scheme to select the preserved values that reduces the memory requirement from $O(l)$ to $O(\sqrt{l})$ at the cost of a single additional forward pass per mini-batch. OpenAI implementation of gradient checkpointing [84] obtains an impressive 10× reduction in memory at the cost of a 20% increase in computation time.

Reversible Layers [25, 26, 35]: As explained above, the back-propagation requires the activations of all intermediate layers, which are either stored in memory during the forward pass or recomputed during the backward pass. As a solution to the latter case, reversible layers allow their activation to be reconstructed exactly from the next layer; therefore, activations must only be stored for one layer and their memory cost becomes independent of the network's depth. More formally, each reversible layer takes as input (x_1, x_2) and outputs (y_1, y_2) such that $y_1 = x_1 + f(x_2)$ and $y_2 = x_2 + g(y_1)$. Each layer's activations are easily reconstructed as $x_2 = y_2 - g(y_1)$ and $x_1 = y_1 - f(x_2)$.

Kitaev et al. [59] used reversible layers in their Transformer, called the Reformer, by combining the attention and feed-forward sub-layers inside a reversible layer. Specifically, $f(\cdot)$ and $g(\cdot)$ were the Attention($\cdot$) and FFN($\cdot$) functions, respectively. The authors observed that reversible layers reduced the memory usage of a 3-layer Transformer without degrading its performance. Nonetheless, reversible layers add numerical errors that accumulate over multiple layers and may degrade the model performance. Therefore, they are not suited for very deep networks.

Gradient checkpointing and reversible layers are very much alike in that they trade computations for memory by recomputing activations during backpropagation. This tradeoff is sometimes necessary: Although computation bottlenecks entail longer running times, memory bottlenecks are critical, as they prevent using the model altogether.

Parameter Sharing: A simple approach to reduce the number of trainable parameters is to impose sets of parameters to be equal in different parts of the network. In other words, the same parameters are used for multiple operations but need to be stored only once in memory. Such a technique is often referred to as parameter sharing, weight tying, or weight replication. As explained in Section 1 and illustrated in Figure 1, recurrent neural networks are built around this idea of parameter sharing to process variable-length sequences. Parameter sharing has also been applied to Transformers. For instance, the Linformer [126] shares projection matrices across heads

Table 2. Multiple Knowledge Distillations of BERT_{BASE}

Model	Compression	Speedup	Mean Relative Performance
BERT _{BASE} [24]	1.0×	1.0×	100%
DistilBERT [99]	1.7×	1.6×	97%
MiniBERT [120]	6.0×	2.6 – 4.3×	97% – 99%
TinyBERT [54]	7.5×	9.4×	97%

Speedups are evaluated on GPUs.

and layers, and the Reformer [59] shares its queries and keys parameters, that is, $\mathbf{W}^Q = \mathbf{W}^K$. Both authors investigated the impact of parameter sharing and concluded that it did not degrade their respective models' performance on their tasks. Lan et al. [62] shared all parameters between layers, which drastically reduced the number of parameters but also decreased the performance by up to 2.5%, on average. They observed that sharing only the attention parameters resulted in a slight drop in performance of 0.7%, on average. The decrease in performance is to be expected, since parameter sharing reduces the number of free parameters, hence the model's capacity.

Pruning [63]: Smaller neural networks are not only faster and lighter, but they are also more likely to generalize better than larger models, because they presumably extract underlying explanatory factors without redundancy. To reduce the model size, weights with a small saliency, that is, whose deletion have a small effect on the loss, may be removed from large models after training. Methods that consider individual weights are said to be *unstructured*, and methods that consider pieces of the network structure such as attention heads or layers are said to be *structured*. Many structured and unstructured pruning schemes have been proposed, several of which have been applied to Transformers. For instance, Sajjad et al. [98] reduced the size of BERT by 40% by dropping complete layers while retaining between 97% and 98% of its original performance, and Michel et al. [79] pruned away between 20% and 40% of BERT attention heads without any significant loss in performance. Recently, the lottery ticket hypothesis has brought a new justification to pruning neural networks. As introduced by Frankle and Carbin [32], the hypothesis states that a “randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.” Prasanna et al. [89] successfully verified this hypothesis on BERT, even noticing that BERT worst subnetworks remain highly trainable. Nonetheless, pruning has two limitations: A large model must be trained, and unstructured pruning schemes produce sparse models unoptimized for modern GPUs and **tensor processing units (TPUs)**.

Knowledge Distillation [2, 43]: The knowledge of a large model or an ensemble of models (teacher) is transferred to a single smaller model (student) by training the student to reproduce the teacher's outputs or its internal behavior. The cumbersome teacher is then discarded, and the student is used at inference time. Given a parameter budget, networks trained with knowledge distillation usually outperform models directly trained on the task. Sanh et al. [99], Tsai et al. [120], and Jiao et al. [54] applied different knowledge distillation schemes on the original BERT [24] to obtain lighter and faster models called DistilBERT, MiniBERT, and TinyBERT, respectively. Table 2 reports their compression, speedup, and performance. Although knowledge distillation achieves impressive compression ratios and performance tradeoffs, a large teacher model still needs to be trained, and the student may perform significantly worse than the teacher. For instance, BERT_{BASE} achieves an accuracy of 52.8% on the CoLA task [129], while DistilBERT and TinyBERT only achieve 32.8% and 44.1%, respectively, according to Jiao et al. [54].

Mixed-precision [80]: Modern GPUs and TPUs perform at least twice as many half-precision (16 bits) float operations as single-precision (32 bits) ones. A popular approach to accelerate training and reduce memory consumption is storing and computing the weights, activations, and

gradients in half-precision. A master copy of the weights is stored in single-precision for numerical stability and minimal performance loss. Thanks to NVIDIA's Automatic Mixed-precision included in some of the most popular deep learning libraries, namely, TensorFlow, PyTorch, and MXNet, using mixed precision can be as simple as adding one line of code. Consequently, we highly recommend mixed precision. Jacob et al. [51] improved over this approach by quantizing both weights and activations as 8-bit integers and biases as 32-bit integers, effectively allowing inference to be performed using integer-only arithmetic. Given a parameter matrix $\mathbf{W}$, N -bit quantization rounds each parameter to one of 2^N codewords corresponding to bins evenly spaced by a scale factor s and shifted by a bias z computed as follows:

$$s = \frac{\max \mathbf{W} - \min \mathbf{W}}{2^N - 1} \quad \text{and} \quad z = \text{round} \left(\frac{\min \mathbf{W}}{s} \right). \quad (10)$$

Each parameter $W_{i,j}$ is quantized to its nearest codeword, and dequantized as:

$$\hat{W}_{i,j} = \left(\text{round} \left(\frac{W_{i,j}}{s} + z \right) - z \right) \times s. \quad (11)$$

To mitigate the performance loss associated with the low-precision approximation, **Quantization Aware Training (QAT)** [51] quantizes the parameters during training. Since quantization is not differentiable, gradients are approximated with a straight-through approximator [7]. Notably, Zafrir et al. [138] quantized all matrix product operations in BERT fully connected and embedding layers during training, reducing the memory footprint by 4× while retaining 99% of the original accuracy on the GLUE [124] and SQuAD [94] tasks. Stock et al. [107] achieved an even higher compression ratio with **iterative product quantization (iPQ)**, which replaces vectors of weights by their assigned centroid and quantization of those centroids. The authors reduced the size of a 16-layer Transformer by 25×, making the model only 14 MB, while retaining 87% of the original performance on the Wikitext-103 [78] benchmark.

While pruning and knowledge distillation achieve faster and lighter models by reducing the number of parameters, mixed precision and quantization instead reduce the number of bits per parameter.

Micro-batching [48]: Increasing model capacity and data throughput are efficient strategies for improving performances in deep learning. However, increasing data throughput requires transferring large mini-batches to the accelerators' memory,⁵ which is also used to store the model. One way to partially avoid the tradeoff between mini-batch size and model size is to use model parallelism. GPipe [48] is a model parallelism library that enables users to distribute a model by grouping layers into cells assigned to accelerators. To avoid the communication bottleneck between accelerators due to the forward and backward operations, the authors proposed a novel batch-splitting algorithm that further splits the mini-batch into micro-batches. As soon as the first accelerator finishes the forward operation of the layers assigned to it for a micro-batch, it sends the result over the communication link and starts processing the next micro-batch. After finishing the last micro-batch's forward operation, the accelerators wait for the first micro-batch's backwards operation results. This waiting time can be used to recompute the forward operation and further reduce memory usage, known as rematerialization. Finally, once the backward operation is completed on the last micro-batch, the algorithm sums all micro-batch's gradients to obtain the mini-batch's gradient (see Figure 6). However, the result is not exact with layers that compute statistics across all mini-batch examples, such as a batch normalization layer [50]. Finally, GPipe is compatible with data parallelism, where multiple mini-batches are processed in parallel.

⁵ An accelerator denotes any device that accelerates computation, such as a graphics or tensor processing unit.

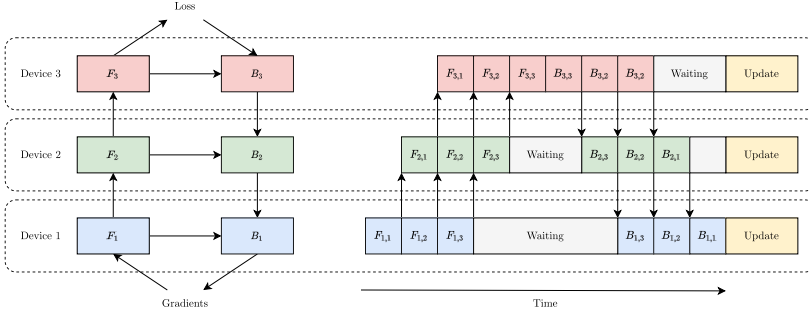


Fig. 6. Micro-batching applied to a model distributed across three devices [48]. F_i and B_i denote the sequential forward and backward operations, respectively, performed by the i th device. Computation on a device may start as soon as the previous device in the computational graph has processed the first micro-batch. Therefore, micro-batching reduces the waiting time of each device at the cost of inter-device communications. Note that the model update is done synchronously at the end.

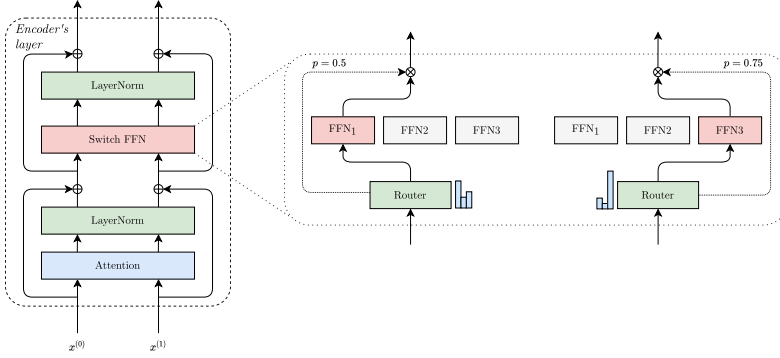


Fig. 7. The computational graph of a single layer of the Switch Transformer's encoder [30]. The Transformer's **feed-forward network (FFN)** has been replaced by a Switch FFN that independently routes each position to an expert. The expert's output is multiplied by the gate value. Note that the computational cost is independent of the number of experts, since a single expert is active for each position.

Huang et al. [48] empirically demonstrated that GPipe allows the maximum Transformer size to scale linearly with the number of accelerators. For instance, a TPU v3 with 16 Gb of memory can only fit a 3-layer Transformer. With GPipe, the same TPU is able to fit 13 layers, while 128 TPUs are able to fit 1,663 layers, which is 127.9 $\times$ more. Additionally, the authors distributed a 48-layer Transformer across 8 TPUs and reported that the training throughput was 4.8 times higher with 32 micro-batches than with a single one.

Mixture of Experts [52]: The core idea is to train multiple networks called experts, each of which specializes only in a subset of the data, and a manager or router, which forwards the input to the corresponding experts. A single network is used in practice, whose layers are composed of multiple subsets of parameters (experts), effectively resulting in a sparsely activated model, as illustrated in Figure 7. Increasing the number of experts keeps the computational cost constant, since the model always selects the same number of experts for each input regardless of the number of experts. Therefore, the **mixture of experts (MoE)** approach allows for massive models and is particularly efficient for distributed systems in which experts are spread across devices. In that case, the number of experts, and therefore parameters, scales with the number of devices available. Despite these advantages, the mixture of experts has not yet been widely adopted, as the

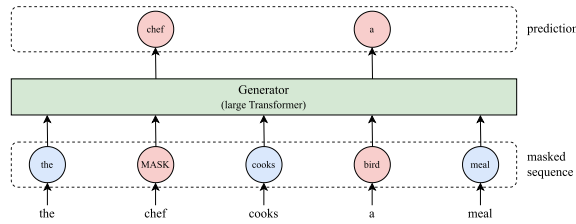


Fig. 8. The masked language model objective [24]. The masked words are depicted in red. The model makes a prediction only for the masked words; thus, MLM is computationally inefficient.

method is complex to deploy in practice. It imposes a communication cost between the devices, a computation cost to select the experts for each input position, and makes training unstable. Recently, Fedus et al. [30] introduced the Switch Transformer based on a carefully crafted mixture of experts. Notably, given a fixed amount of computation per input position, the Switch Transformer reached the same quality threshold as a vanilla Transformer five times faster (wall-clock time), on average. Additionally, when trained further, the Switch Transformer outperformed the vanilla baseline. However, this approach assumes that multiple regimes with distinct input to output relations produce the data.

Difficult tasks often require large models to achieve the desired performance. However, such models require powerful and expensive accelerators. Both micro-batching and the mixture of experts offer an alternative to train such models on many relatively weak and inexpensive GPUs at the cost of complex implementation.

Sample-efficient Objective [19]: Large neural networks, especially Transformers, benefit from being pre-trained with an unsupervised objective before being fine-tuned on the task of interest, also called the downstream task. The core idea is to leverage large unlabelled datasets that are easy to automatically collect to learn the data underlying explanatory factors and ultimately improve the model performance. Concretely, pre-training initializes the network’s weights in a “good” region of space. As pre-training of large models is often more compute-intensive than fine-tuning, researchers regularly share pre-trained models to facilitate their adoption. Most notably, Hugging Face [132] is an open-source library that contains an extensive collection of pre-trained Transformers under a unified API. Nonetheless, researchers must sometimes pre-train models themselves due to the peculiar nature of the data or the problem at hand. In that case, a sample-efficient objective will reduce the computation required.

Recently, Devlin et al. [24] popularized the Cloze procedure [117] for pre-training under the name of **masked language model (MLM)**, which independently estimates the probability of masked words given the rest of the sequence. Practically, 15% of the words are randomly selected, of which 80% are masked, 10% are replaced by a random word, and 10% are left unchanged. This task is analogous to the reconstruction of corrupted input. Figure 8 illustrates the masked language model objective.

Clark et al. [19] introduced the replaced token detection objective to speed up pre-training; a small network (generator) first generates a plausible alternative for each masked word, then the large model (discriminator) predicts whether each word has been replaced (see Figure 9). While the masked language model makes a prediction only for the masked works, the replaced token detection makes a prediction for every word. Therefore, the latter is more computationally efficient than the former; in other words, less pre-training computations are required to achieve the same performance on downstream tasks. Additionally, the authors reported that the representations learned with their objective outperformed those learned with MLM given the same model size,

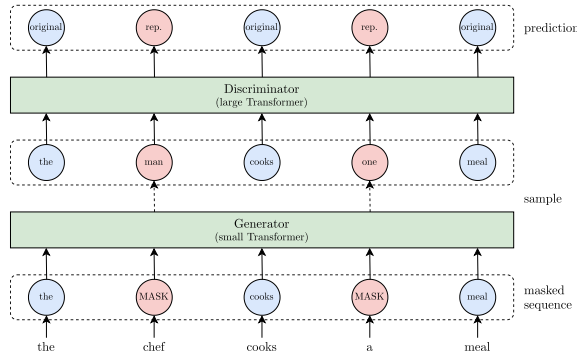


Fig. 9. The replaced token detection objective [19]. A plausible alternative of each masked word is sampled from a small generator network. Then, a discriminator predicts whether each word has been replaced.

data, and computation. Most notably, they were able to outperform GPT on the GLUE benchmark with 30× fewer computations.

Parameter Initialization Strategies: Optimizing deep networks is challenging in part because of the considerable influence of the initial point on the iterative process. Notably, the initial point determines whether the algorithms converge at all and, if it does converge, the speed at which it converges as well as the quality of the solution [36]. Transformers are notoriously difficult to train, typically requiring carefully tuned optimizers with adaptive learning rates, learning rate schedulers, and large batches. Even then, convergence is not guaranteed. Consequently, Liu et al. [73] and Huang et al. [47] concurrently proposed initialization schemes for the Transformer that promise a smoother and faster optimization as well as better generalization performances.

Liu et al. [73] identified an amplification effect that significantly influences training: each layer heavily depends on its residual branch,⁶ making the optimization unstable as it amplifies small parameter perturbations. Ultimately, the amplification effect may produce a notable change in the Transformer’s output. Nonetheless, the authors observed that heavy dependencies on the residual branches are necessary to unlock the Transformer’s potential and achieve better results. To mitigate the amplification effect, Liu et al. [73] introduced the Adaptive Model Initialization strategy, or Admin, which controls the dependency on the residual connections in the early stage of training with a new parameter ω . Formally, the i th sub-layer output is given by

$$X_i = \text{LayerNorm}(f_i(X_{i-1}) + X_{i-1} \odot \omega_i), \quad (12)$$

where $f_i(X)$, X_{i-1} , and X_i , denote the function, input, and output of the i th sub-layer, respectively. Although this is equivalent to rescaling some model parameters, the authors observed that rescaling leads to unstable training in half-precision.

The proposed initialization strategy requires three steps. First, the model parameters are initialized with a standard method such as the Xavier initialization [34] and the Admin parameter ω with ones. Then, one or a small number of mini-batches are forward propagated without updating the parameters and record the output variance of each residual branch $\text{Var}[f_i(X_{i-1})]$. Finally, the Admin parameter is initialized as $\omega_i = \sqrt{\sum_{j < i} \text{Var}[f_j(X_{j-1})]}$. Once the model has been trained, ω may be discarded.

The amplification effect is, however, not the only mechanism that makes Transformers notoriously difficult to train. Huang et al. [47] addressed two other issues: (i) Transformers are typically

⁶For a residual block $f(x) + x$, the residual branch refers to $f(x)$ and the skip connection, shortcut connection, or residual connection refers to x .

trained with optimizers that rely on adaptive learning rates, as conventional SGD fails to train them effectively. However, adaptive learning rates have a problematically large variance in the early stages of optimization, resulting in convergence issues [72]; and (ii) the magnitude of the error signal propagated through LayerNorm is inversely proportional to the magnitude of the input [135]. Specifically, the norm of the layer normalization gradient is proportional to:

$$\left\| \frac{\partial \text{LayerNorm}(\mathbf{x})}{\partial \mathbf{x}} \right\| = O\left(\frac{\sqrt{d}}{\|\mathbf{x}\|}\right). \quad (13)$$

Consequently, if the input norm $\|\mathbf{x}\|$ is larger than $\sqrt{d}$, then backpropagating through layer normalization reduces the gradient magnitude for layers closer to the input. As a solution to both problems, Huang et al. [47] proposed an initialization strategy called T-Fixup that restricts the magnitude of the updates in the early stages of training, thus mitigating the vanishing gradient issue while eliminating the need for layer normalization and warmup.

While Liu et al. [73] and Huang et al. [47] claim faster convergence, they did not report the improvement.

Architecture Search: One of the most challenging goals in deep learning is to automatically design networks. Indeed, the problem of finding architectures that achieve the best performance with the fewest operations and lowest memory footprint in a discrete search space is an NP-hard combinatorial optimization problem. Over the years, multiple approaches to **Neural Architecture Search (NAS)** have been proposed, including reinforcement learning [141], evolutionary algorithms [95], and bilevel optimization [71]. Notably, Zoph et al. [142] demonstrated that NAS is able to surpass human-designed architectures on ImageNet by 1.2% top-1 accuracy while using 28% fewer computations. Nonetheless, neural architecture search methods are computationally expensive, as they usually require training each candidate model from scratch. As a solution, Pham et al. [88] proposed **Efficient NAS (ENAS)**, which constrains all candidates to be subgraphs of a single computational graph, that is, to share parameters. Therefore, the ENAS's controller decides which operations are activated and relies on the models' ability to adapt, similarly to dropout [106]. Efficient NAS reduces the search computational budget by 1,000× over the original NAS [141]. Alternatively, Liu et al. [71] proposed the **Differentiable Architecture Search (DARTS)**, which casts the NAS problem as a differentiable bilevel optimization problem. The first level consists of a continuous relaxation of the discrete search space using a Softmax function over a list of candidate operations, and the second level involves the model's weights. However, the bilevel formulation requires training the weights to convergence to evaluate the architecture gradient. To avoid this substantial cost, the authors made the approximation of taking a single gradient step of the weights for one gradient step of the architecture parameters. The authors obtained comparable performances to non-differentiable NAS methods on ImageNet in the mobile setting using only 4 GPU-days, compared to 3,150 for evolutionary algorithms [95] and 2,000 for NAS [142]. Differentiable Architecture Search obtained comparable results to ENAS with a similar computational budget. We refer the reader to the Elsken et al. [29] survey for further detail on architecture search methods.

Nevertheless, neural architecture search methods are challenging to apply on Transformers due to the memory requirements and training time. Therefore, recent works introduced methods better suited for the Transformer. So et al. [105] modified the tournament selection evolutionary architecture search [95] with **Progressive Dynamic Hurdles (PDH)**, which dynamically allocates resources to more promising architectures according to their performances. With PDH, the authors optimized transformer architectures directly on the WMT'14 En-De task [9], which requires 10 hours of computation on a Google TPU v2 for the base Transformer model. Training directly on this dataset is essential, since the authors did not find a smaller surrogate dataset that transfers

well, such as CIFAR-10 for ImageNet. The Evolved Transformer matched the vanilla Transformer's performance with only 78% of its parameters. Recently, Tsai et al. [119] profiled the Transformer's components on a TPU v2 and observed that some mechanisms substantially impact inference time: attention queries, keys, and values dimensions, width and depth of feed-forward layers, number of attention heads, and layer normalization mean computation. By decomposing these components into building blocks and using binary variables, the authors perform a one-shot search for both the architecture and the parameters with a single loss. They optimized this loss with gradient descent on a continuous relaxation of the binary variables and used policy gradient algorithm. Tsai et al. [119] were able to make miniBERT $1.7\times$ faster with a performance drop smaller than 0.3%. Compared to the original BERT, this is 33 to $36\times$ faster.

Neural architecture search is a promising tool to design lighter and faster Transformers automatically. Nonetheless, NAS imposes a high computational and memory cost, which may be avoided by carefully engineering the architecture instead. For instance, the Lite Transformer [133] leverages the **Long-Short Range Attention (LSRA)**, where a convolutional layer is applied in parallel to the self-attention to learn the local dependencies separately. The carefully handcrafted Lite Transformer outperforms the Evolved Transformer [105] for the mobile NLP setting while requiring about $14,000\times$ less GPU time.

Conditional Computing [6]: Although large models are necessary for hard examples, smaller models are likely to perform as well, if not better, on simpler ones. For instance, many words such as “car” are easy to translate, while a few such as “can” require careful consideration of the context.⁷ As of this survey's writing, most architectures apply a fixed number of operations to all examples regardless of their difficulty. A more efficient approach would be to reduce the amount of computation for simple examples. As a solution, Bengio [6] introduced conditional computing, which dynamically adapts the model's computational graph as a function of the input.

One way to implement conditional computing is with a mixture of experts, as introduced previously. In that case, only a subset of the parameters is used for a given input, making the computational graph sparse and the computation time almost constant with respect to the model size. Another approach consists of keeping the number of parameters constant and letting the model adjust its computation time separately for each input (according to the input's value). This approach is called **Adaptive Computation Time (ACT)** [38] and uses a recurrent mechanism to transform the representations until a halting probability exceeds a given threshold. The model learns to control this probability to minimize both the prediction error and the number of iterations, called the *ponder cost*, which prevents the model from using an infinite amount of computation before making a prediction. One shortcoming of the Adaptive Computation Time is its sensitivity to the ponder cost, which controls the tradeoff between speed and accuracy.

Dehghani et al. [23] applied ACT to a Transformer with a recurrent mechanism for the architecture's depth. To implement this mechanism, the authors defined encoder and decoder blocks similar to the original Transformer, except that each block is recurrent, sending its output back as its input until the ponder cost becomes too high. Note that a fixed number of recurrent steps is equivalent to a Transformer with tied parameters across all layers. With this new architecture called Universal Transformer, the authors claimed that it is computationally universal (Turing-complete) given enough memory. This property may help Transformers generalize to sequences longer than the ones seen during training. The authors obtained state-of-the-art results on algorithmic and language understanding tasks. ACT and the Universal Transformer apply the same layers iteratively, which may not be sufficiently flexible. Elbayad et al. [28] addressed this

⁷Depending on the context, the word “can” has various meanings, including “be able to,” “may,” “jail,” and “metal container.” See <https://www.wordreference.com/definition/can>.

limitation with the **Depth-Adaptive Transformer (DAT)**, which applies different layers at every depth. The DAT matches the performance of a well-tuned Transformer baseline while reducing the computation by up to 76%. However, the authors did not provide a comparison between the Universal Transformer and DAT.

In the same way that complex examples may require more computations, some may require access to a longer context. As a solution, Sukhbaatar et al. [111] dynamically adjusted the attention span, that is, the context length, by learning to mask the compatibility scores depending on the input. Their approach achieved state-of-the-art on text8 and enwik8 [77] while requiring significantly fewer computations. Alternatively, Li et al. [65] introduced the **Decoder-end Adaptive Computation Steps (DACS)**, which monotonically computes halting probabilities along with the encoder states and stops the decoder computations to produce an output when the accumulation of probabilities exceeds a given threshold. In other words, each decoder step only looks at the necessary information as measured by the halting probabilities instead of looking at the entire input sequence.

4 SPECIALIZED APPROACHES

Since the Transformer's quadratic complexity comes from the attention mechanism, most specialized methods rely on a fast and light approximation of the original full attention. As will be explained in greater detail in the rest of this section, the attention weight matrix is dominated by a few large values and is approximately low-rank. These observations justify two distinct lines of work: sparse attention and factorized attention. Alternatively, the complexity may be reduced without altering the original attention mechanism and thus the Transformer's capacity by directly modifying the network's architecture. Let us first investigate the approaches that rely on sparse attention.

Note that some approaches only consider autoregressive tasks, such as the left-to-right language model, and in that case, the connectivity matrix is lower triangular, as it is not permitted to attend to future positions. Whenever possible, such works have been extended to the more general case where attending to future positions is allowed to ease the comparison between the different approaches.

4.1 Sparse Attention

Due to the exponential nature of the Softmax, only a few positions are strongly attended to. Consequently, a conceptually simple way of reducing the Transformer's complexity is to make the matrix QK^T sparse,⁸ in other words, to only allow each position to attend to a subset of the positions. Let us investigate sparse patterns that are (i) fixed and random, (ii) learned and adaptive, and (iii) identified with clustering and locality sensitive hashing.

Fixed and Random Sparse Patterns [5, 16, 41, 66, 90, 125, 139]: One of the first models to consider fixed sparse patterns is the Star-Transformer introduced by Guo et al. [41], which reduced the complexity from quadratic to linear by only allowing attention between adjacent positions. To preserve the Transformer's ability to model long-term dependency, the authors relied on a single global token. Global tokens, also known as shared relay nodes, can attend to every position, and every position can attend to global tokens. Let us assume that the global token is located at position 0. The i th output position is allowed to attend to every input position if $i = 0$, otherwise, it is allowed to attend to the j th input positions for $j = 0$ and if $i - 1 \leq j \leq i + 1$. Figure 10 illustrates the Star-Transformer attention pattern.

⁸Since the matrix QK^T is passed through a Softmax function, the masked values are set to minus infinity, effectively setting their contribution to $e^{-\infty} = 0$.

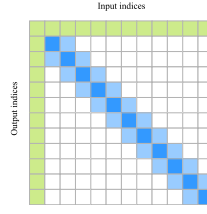


Fig. 10. The connectivity matrices of the Star-Transformer [41].

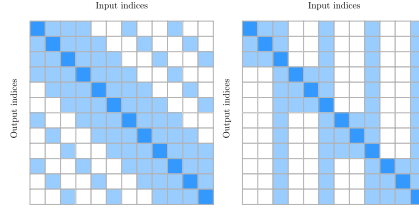


Fig. 11. The connectivity matrices of the Sparse Transformer Child et al. [16]. (Left) Strided attention with a stride of 3. (Right) Fixed attention with a stride of 3 and $c = 1$.

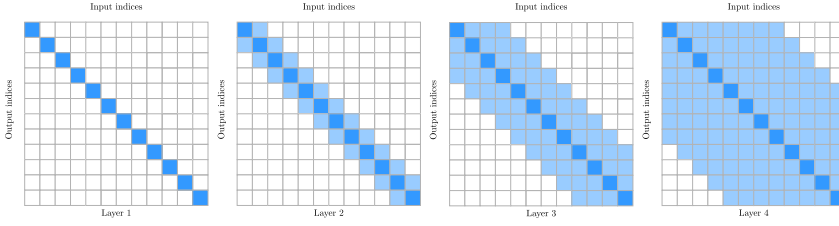


Fig. 12. The connectivity matrices of the Cascade attention [125] for the first four layers with a base window $b = 1$ and a cardinal number $m = 2$. For instance, the window size of the third layer ($l = 2$) is equal to $2 \times b \times m^l - 1 = 7$.

Concurrently, Child et al. [16] introduced the Sparse Transformer, which reduced the complexity to $O(n\sqrt{n})$ with two different sparse attention patterns: strided and fixed. Strided attention allows the i th output position to attend to the j th input position if one of the two following conditions is satisfied: $(i + s) > j > (i - s)$ or $(i - j) \bmod s = 0$, where the stride s is chosen to be close to $\sqrt{n}$. Similarly, fixed attention allows i to attend to j if one of the two following conditions is satisfied: $\text{floor}(j/s) = \text{floor}(i/s)$ or $(j \bmod s) \geq (s - c)$, where c is an hyperparameter. Figure 11 illustrates the strided and fixed attention patterns.

Alternatively, Wang et al. [125] introduced the Cascade Transformer, which relies on sliding window attention whose size grows exponentially with the number of layers. More specifically, the number of cascade connections at the layer l is equal to $2 \cdot b \cdot m^l - 1$, where b is the base window size and m is the cardinal number, therefore reducing the complexity to $O(n \cdot b \cdot m^l)$. Cascade attention is well suited for shallow networks, but its complexity tends to that of the full attention in deep networks, as depicted by the connectivity matrices in Figure 12.

Li et al. [66] introduced the LogSparse-Transformer for forecasting fine-grained time series with strong long-term dependencies. The LogSparse-Transformer relies on the eponym attention that allows the i th output to attend to the j th inputs for $j \in \{-2^{\lfloor \log_2 i \rfloor}, i - 2^{\lfloor \log_2 i \rfloor - 1}, \dots, i - 2^1, i - 2^0, i, i + 2^0, i + 2^1, \dots, i + 2^{\lfloor \log_2(n-i) \rfloor - 1}, i + 2^{\lfloor \log_2(n-i) \rfloor}\}$ where $\lfloor \cdot \rfloor$ denotes the floor operation and N denotes the sequence length. Figure 13 illustrates the connectivity matrix of the LogSparse

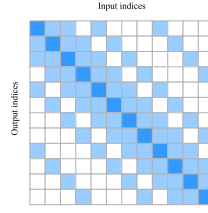


Fig. 13. The connectivity matrix of the LogSparse attention Li et al. [66].

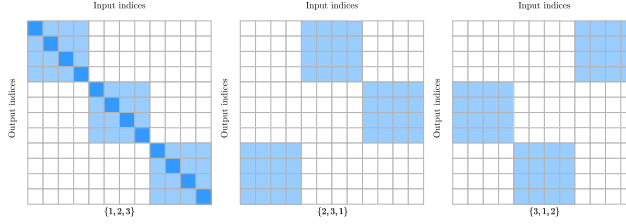


Fig. 14. The connectivity matrices of the block-wise attention [90] for $n_b = 3$ blocks. The corresponding permutations are written below the connectivity matrices.

attention. Since only $O(\log n)$ positions are attended to by each of the n positions, the complexity of the LogSparse attention is $O(n \log n)$. Additionally, the authors proposed two alternatives: (1) to allow the i th output to attend to the first k input positions, after which the LogSparse attention is resumed, and (2) to divide the input sequence into subsequences and to apply the LogSparse attention on each of them.

Qiu et al. [90] introduced BlockBERT, which relies on the block-wise attention: The input sequence is split into n_b non-overlapping blocks, and positions in block i are only allowed to attend to positions in block $\pi(i)$, where π denotes a permutation. The author chose to generate the permutations by simply shifting the positions. For instance, the possible permutations of $\{1, 2, 3\}$ are $\{1, 2, 3\}$, $\{3, 1, 2\}$, and $\{2, 3, 1\}$. The permutation $\{2, 3, 1\}$ means that the first block attends to the second block, the second block attends to the third block, and the third block attends to the first block. In the multi-head setting, a different permutation⁹ is assigned to each head. More formally, the output position i is only allowed to attend to input j if the following condition is satisfied:

$$\pi \left(\left\lfloor \frac{(i-1)n_b}{n} + 1 \right\rfloor \right) = \left\lfloor \frac{(j-1)n_b}{n} + 1 \right\rfloor. \quad (14)$$

Figure 14 illustrates the connectivity matrix of the block-wise attention where a sequence of length $n = 12$ is split into $n_b = 3$ blocks. Although the block-wise attention reduces the memory and computational cost by a factor n_b , the complexity remains quadratic with respect to the sequence length.

Beltagy et al. [5] introduced the Longformer, which further reduces the complexity to $O(n)$ using a combination of sliding window and global attentions (see Figure 15). The assumption behind the sliding window attention is that the most useful information is located in each position's neighborhood. The sliding window attention is limited in that it requires $O(\sqrt{n})$ layers to model long-range dependencies. Thus, a few preselected tokens have a global attention: They can attend to every position and be attended by every position. Consequently, the maximum path length between any two positions is equal to 2. Zaheer et al. [139] introduced BigBird, which also achieves a linear

⁹Note that if the number of heads is greater than the number of permutations, then multiple heads must be assigned the same permutation.

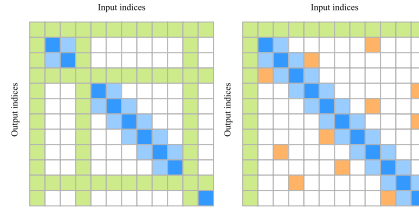


Fig. 15. The connectivity matrices of two sparse attention schemes. (Left) Longformer [5]. (Right) BigBird [139]. The attention is the combination of sliding window attention (blue), global attention (green), and random attention (orange).

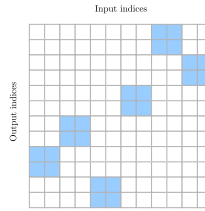


Fig. 16. The connectivity matrix of the sparse Sinkhorn attention [114].

complexity using a combination of random, sliding window, and global attentions (see Figure 15). BigBird has two configurations that the authors referred to as **internal transformer construction (ITC)** and **extended transformer construction (ETC)**. Similarly to the Longformer, the former uses existing positions for global attention, while the latter uses additional tokens, increasing the model's capacity and performance. Interestingly, the extra location of ETC may be seen as a form of memory. The authors proved that their sparse factorization preserves the theoretical properties of Transformers with the full attention: The model is both a universal approximator of sequence functions and Turing complete. However, BigBird without random attention outperformed BigBird with it in most of their experiments.

Learned and Adaptive Sparse Patterns [20, 104, 114]: Fixed and random patterns are hand-crafted and may not be suitable for the data and task at hand. One may instead learn the relevant patterns and adapt them based on the content.

To increase the flexibility of the block-wise attention, Tay et al. [114] introduced the sparse Sinkhorn attention, which is equivalent to the block-wise attention whose keys have been sorted in a block-wise fashion. In other words, the permutations are learned. More specifically, the sparse Sinkhorn attention transforms the input sequence $X \in \mathbb{R}^{n \times d}$ into $X' \in \mathbb{R}^{n_b \times d}$, where n_b is the number of blocks and where X'_i is equal to the sum of the input in that block. A simple feed-forward network then learns a mapping $R_i \in \mathbb{R}^{n_b}$ from the i th block X'_i to all blocks. To obtain a sorting matrix from $R \in \mathbb{R}^{n_b \times n_b}$, that is, a matrix comprising only 0s and 1s, and whose rows and column sum to one, the rows and columns are iteratively normalized. The sorting matrix is then used to permute the keys, effectively learning which block to attend (see Figure 16). The sparse Sinkhorn attention reduces the complexity to $O(n_b^2)$. Nonetheless, since the block size is constant in the original paper, the complexity remains quadratic with respect to the sequence length. Additionally, the authors proposed a truncated version of the sparse Sinkhorn attention, which selects a few keys after sorting them, further reducing the complexity to $O(n)$.

Recently, Shi et al. [104] put under the microscope the attention patterns learned by BERT [24] and observed that the diagonal elements are less important compared to other positions, that is, they contribute the least to the output, while neighborhood positions and special tokens are prominent. To confirm their observations, they dropped the diagonal element in BERT's attention such

that each position is not allowed to attend to itself and noted that the performance remains comparable to the original model. Additionally, they observed that models for different tasks have various degrees of redundancy and hence can achieve various sparsity levels before significantly dropping performance. Consequently, Shi et al. [104] proposed to learn sparsity patterns for each task in an end-to-end fashion with the **Differentiable Attention Mask (DAM)** algorithm. Let us denote the attention score between the i th output position (query) and j th input position (key) as $\alpha_{i,j}$. They proposed to compute the attention mask $M_{i,j}$ as the Gumbel – Sigmoid [76] of the attention score $\alpha_{i,j}$:

$$M_{i,j} = \text{Gumbel} - \text{Sigmoid}(\alpha_{i,j}) = \text{Sigmoid}\left(\frac{\alpha_{i,j} + G_1 - G_2}{\tau}\right), \quad (15)$$

where G_1, G_2 are independent Gumbel noises $G_k = -\log(-\log(U_k))$ generated from a uniform distribution $U_k \sim \mathcal{U}(0, 1)$ and where τ is a temperature hyperparameter. Note that the Gumbel – Sigmoid becomes binary as τ approaches 0. A penalty term $\lambda \|M\|_1$ is added to the loss to control the tradeoff between performance and sparsity. The resulting model called SparseBERT achieved 91.2% sparsity while maintaining an average score of 80.9% on GLUE, i.e., only 3% lower than the full BERT. Such an approach deviates from previous sparse attention whose patterns have been manually handcrafted. To avoid learning completely unstructured sparsity patterns, the authors proposed to enforce the first and last row/column of the attention mask to be active and all positions on each line parallel to the diagonal to share their mask parameters.

As mentioned above, due to the exponential nature of the Softmax, most positions are lightly attended to. In other words, most attention weights are small but non-zero. Instead, Correia et al. [20] introduced the Adaptively Sparse Transformer that replaces the Softmax by the α – entmax function, a differentiable generalization of the Softmax that pushes small weights to be exactly zero. Formally, the α – entmax function is defined as:

$$\alpha - \text{entmax}(\mathbf{z}) = \underset{\mathbf{p} \in \Delta^d}{\operatorname{argmax}} \mathbf{p}^\top \mathbf{z} + H_\alpha^T(\mathbf{p}), \quad (16)$$

where $\Delta^d = \{\mathbf{p} \in \mathbb{R}^d : \sum_i p_i = 1\}$ and, for $\alpha \geq 1$, H_α^T is the Tsallis continuous family of entropies:

$$H_\alpha^T(\mathbf{p}) = \begin{cases} \frac{1}{\alpha(\alpha-1)} \sum_j (p_j - p_j^\alpha), & \alpha \neq 1 \\ -\sum_j p_j \log p_j, & \alpha = 1. \end{cases} \quad (17)$$

The authors showed that the solution to the Equation (16) is

$$\alpha - \text{entmax}(\mathbf{z}) = [(\alpha - 1)\mathbf{z} - \lambda \mathbf{1}]_+^{\frac{1}{\alpha-1}}, \quad (18)$$

where $[\]_+$ denotes the ReLU function, $\mathbf{1}$ denotes the vector of ones, and λ is the Lagrange multiplier corresponding to the $\sum_i p_i = 1$ constraint.

Interestingly, when $\alpha = 1$, the α – entmax is equivalent to the Softmax, and the attention is dense, and when $\alpha > 1$, the output is permitted to be sparse. In their experiments, a scalar parameter $a_{i,j}$ is learned for the j th attention head of the i th layer, and $\alpha_{i,j}$ is computed as:

$$\alpha_{i,j} = 1 + \text{sigmoid}(a_{i,j}) \in]1, 2[. \quad (19)$$

Nonetheless, the Adaptively Sparse Transformer computes the attention score for each pair of queries and keys. Consequently, the sparsity cannot be leveraged to improve the memory and computation, resulting in a model that is 25% slower than the original Transformer in terms of tokens per second.

As of this survey's writing, unstructured sparse attention (whether fixed, random, or learned) does not benefit from efficient implementations and therefore cannot result in memory and computational improvements. Nonetheless, there are exciting researches in that direction, as noted by

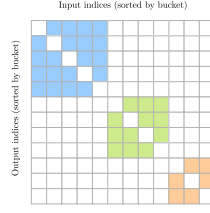


Fig. 17. The connectivity matrix of the Reformer [59]. Queries and keys are bucketed using LSH, then sorted by their bucket. Therefore, the i th row of the connectivity matrix may not correspond to the i th position in the input sequence. Units can only attend other units in the same bucket, but not themselves because queries and keys are equal. The color represents buckets.

Hooker [45]. In contrast, some structured sparsity patterns benefit from efficient implementations. Recently, NVIDIA introduced its Ampere architecture that efficiently compresses 2:4 structured sparsity on rows, that is, two non-zero values in every four entries.

Clustering and Locality-sensitive Hashing [59, 96]: The Softmax function is dominated by the largest values, that is, by the keys and queries that have the largest dot product. Therefore, the attention may be approximated by only comparing the most similar keys and queries. Although this approach is a form of adaptive sparsity, as the patterns depend on the data, they are presented separately due to their conceptual difference.

Kitaev et al. [59] introduced the Reformer, which selects the set of keys that the query can attend to by grouping them with an angular multi-round **locality-sensitive hashing (LSH)**. Such hashing scheme has a high probability of assigning the same value to similar vectors. Formally, queries and keys are shared ($Q = K$) and bucketed using b hash values obtained as follows:

$$\mathbf{p} = [\mathbf{x}^\top \mathbf{R}; -\mathbf{x}^\top \mathbf{R}], \quad (20)$$

$$h(\mathbf{x}) = \underset{i}{\operatorname{argmax}}(p_i), \quad (21)$$

where $;$ denotes the concatenation operation and where $\mathbf{x} \in \mathbb{R}^d$ is a query/key and $\mathbf{R} \in \mathbb{R}^{d \times b/2}$ is a random rotation matrix. Output positions are only allowed to attend to input positions that are in the same bucket (see Figure 17). They are, however, not allowed to attend to themselves, because the dot product of a vector with himself will almost always be greater than the dot product with other positions.

The authors chose a constant bucket size l_B , resulting in a number of buckets $n_B = n/l_B$. The attention complexity is $O(n_B \times l_B^2)$, which simplifies as $O(n)$. This does not take into account the computation of the hash values for each position. As only $\log n_B$ bits are required to encode n_B buckets, the complexity of computing hash values is given by $O(n \log n_B)$, which simplifies as $O(n \log n)$. Consequently, the complexity of the Reformer's attention is $O(n \log n)$.

The **Maximum Inner Product Search (MIPS)** problem is the task of searching for the vector K_j in $K = \{K_1, K_2, \dots, K_n\}$ that maximizes the dot product with a given vector Q_i . Note that the MIPS problem is particularly useful for the attention mechanism, as $Q_i^\top K_j$ is directly proportional to the contribution of the j th value for the i th attention's output. There are multiple approaches to approximately solve this problem, including tree-based and LSH-based. When the norm of every K_j is constant, the problem is equivalent to the **Nearest Neighbor Search (NNS)**. Motivated by this observation and to avoid the computational cost of learning sparsity patterns, Roy et al. [96] proposed the Routing Transformer that relies on an online mini-batch version of k -means and a set of centroids learned along the rest of the parameters. Like the Reformer, queries can only attend to keys from the same cluster, inducing an adaptive or content-based sparsity pattern.

4.2 Factorized Attention

Wang et al. [126] demonstrated that the attention matrix $\text{Softmax}(\mathbf{QK}^\top/\sqrt{d})$ is approximately low rank. Consequently, another approach to reduce the Transformer's complexity is to approximate the attention by factorizing it into the product of two matrices with lower dimensions.

Low-rank Factorization [113, 126, 136]: Wang et al. [126] introduced the Linformer, a linear complexity model that approximates the attention with a low-rank factorization by first projecting each key to a lower dimension before performing the dot product, thereby saving time and memory. Formally, the low-rank attention is given by:

$$\text{Attention}(\mathbf{X}) = \underbrace{\text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)}_{n \times n} \underbrace{\mathbf{V}}_{n \times d} \approx \underbrace{\text{Softmax}\left(\frac{\mathbf{Q}(\mathbf{EK})^\top}{\sqrt{d}}\right)}_{n \times k} \underbrace{\mathbf{FV}}_{k \times d}, \quad (22)$$

where $\mathbf{E}, \mathbf{F} \in \mathbb{R}^{k \times n}$, with $k \ll n$, are two linear projection matrices learned during training. The authors showed that $\mathbf{E}$ and $\mathbf{F}$ could be shared across heads and layers with virtually no performance penalty.

Tay et al. [113] introduced a family of models called Synthesizers that learn the compatibility scores without computing the pairwise dot products between the queries and keys. For instance, the Dense Synthesizer learns the compatibility scores with a simple position-wise feed-forward network that projects each of the n rows of $\mathbf{X}$ from $\mathbb{R}^{1 \times d}$ to $\mathbb{R}^{1 \times n}$:

$$F(\mathbf{X}_i) = \max(0, \mathbf{X}_i \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2, \quad (23)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d}$ and $\mathbf{W}_2 \in \mathbb{R}^{d \times n}$. Finally, the attention is given by:

$$\text{Attention}(\mathbf{X}) = \text{Softmax}(F(\mathbf{X}))G(\mathbf{X}), \quad (24)$$

where $G(\cdot) : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times d}$ is a projection of the input akin to the values. To improve the efficiency, the authors proposed the Factorized Dense Synthesizer, which first projects the input $\mathbf{X}$ with two feed-forward networks:

$$\mathbf{A} = F_A(\mathbf{X}) \in \mathbb{R}^{n \times a} \quad \text{and} \quad \mathbf{B} = F_B(\mathbf{X}) \in \mathbb{R}^{n \times b}, \quad (25)$$

such that $a \times b = n$. Then, two tiling functions $H_A(\cdot) : \mathbb{R}^{n \times a} \rightarrow \mathbb{R}^{n \times (a \cdot b)}$ and $H_B(\cdot) : \mathbb{R}^{n \times b} \rightarrow \mathbb{R}^{n \times (b \cdot a)}$ are applied to $\mathbf{A}$ and $\mathbf{B}$, respectively. Note that a tiling function simply repeats a vector multiple times. Finally, the attention of the Factorized Dense Synthesizer is given by:

$$\text{Attention}(\mathbf{X}) = \text{Softmax}(H_A(\mathbf{A})H_B(\mathbf{B})^\top)G(\mathbf{X}). \quad (26)$$

Additionally, the authors proposed a baseline called the Factorized Random Synthesizer, whose compatibility scores are independent of the input. Formally, the Factorized Random Synthesizer's attention is given by:

$$\text{Attention}(\mathbf{X}) = \text{Softmax}(\mathbf{R}_1 \mathbf{R}_2^\top)G(\mathbf{X}), \quad (27)$$

where $\mathbf{R}_1, \mathbf{R}_2 \in \mathbb{R}^{n \times k}$ are two low-rank matrices learned during training. Although the Synthesizers eliminate the need to compute the pairwise dot products, which speed up the model in practice, the complexity remains quadratic with respect to the sequence length.

The Nyströmformer [136] relies on the Nyström method to generate a low-rank approximation of the Softmax matrix. However, applying the Nyström method directly to the Softmax would require to compute the $\mathbf{QK}^\top$ product, which requires $O(n^2)$ computations and memory. As a solution, the Nyströmformer creates two subsets $\tilde{\mathbf{K}}$ and $\tilde{\mathbf{Q}}$ of columns, called landmarks, from $\mathbf{K}$ and $\mathbf{Q}$, respectively. The authors applied the segment-means approach, which computes the landmarks

as the averages over predefined spans of keys and queries. Let S_{AB} denotes $\text{Softmax}(AB^\top / \sqrt{d})$ for any matrix A and B . The Nyströmformer approximates the Softmax matrix as:

$$\text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) \approx S_{Q\tilde{K}} S_{\tilde{Q}\tilde{K}}^+ S_{\tilde{Q}\tilde{K}}, \quad (28)$$

where the superscript $+$ denotes the Moore-Penrose inverse typically computed with the **singular value decomposition (SVD)**. Since the SVD is inefficient on GPU, the authors relied on an iterative method that approximate $S_{\tilde{Q}\tilde{K}}^+$ as Z^+ . Finally, the Nyströmformer's attention is given by:

$$\text{Attention}(X) \approx S_{Q\tilde{K}} Z^+ S_{\tilde{Q}\tilde{K}} V, \quad (29)$$

which can be efficiently encoded in a computational graph.

Provided that the number of landmarks is constant and much smaller than the sequence length, the Nyströmformer complexity is $O(n)$. Depending on the number of landmarks and the sequence length, the authors reported substantial gains over the Linformer and Longformer on the masked language model and sentence order prediction objectives. Additionally, the representations learned by the Nyströmformer appear to transfer as well as BERT to different NLP tasks. Nonetheless, a more extensive evaluation of the Nyströmformer remains necessary.

Kernel Attention [18, 56]: A kernel $K(\cdot, \cdot)$ is a function that takes two vectors as arguments and returns the product of their projection by a feature map $\phi(\cdot)$:

$$K(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x})^\top \phi(\mathbf{y}). \quad (30)$$

Katharopoulos et al. [56] interpreted the Softmax as a kernel, decomposed it as an inner product in the right space, and rearranged the computations in a clever way to reduce the complexity. More specifically, the self-attention of a given query Q_i may be rewritten using a mapping $\phi(\cdot)$:

$$\text{Softmax}(Q_i^\top K^\top) \mathbf{V} = \frac{\sum_{j=1}^n \exp(Q_i^\top K_j) V_j}{\sum_{j=1}^n \exp(Q_i^\top K_j)} = \frac{\sum_{j=1}^n \phi(Q_i)^\top \phi(K_j) V_j}{\sum_{j=1}^n \phi(Q_i)^\top \phi(K_j)} = \frac{\phi(Q_i)^\top \sum_{j=1}^n \phi(K_j) V_j}{\phi(Q_i)^\top \sum_{j=1}^n \phi(K_j)}, \quad (31)$$

where the scaling factor $\sqrt{d}$ has been omitted for the sake of readability. The authors noted that $\sum_{j=1}^n \phi(K_j) V_j$ and $\sum_{j=1}^n \phi(K_j)$ must only be computed a single time, therefore reducing the complexity from quadratic to linear both in terms of memory and computation. The vectorized formulation of the numerator makes it simpler to see:

$$\underbrace{\phi(Q)}_{n \times p} \left(\underbrace{\phi(K)^\top}_{p \times n} \underbrace{V}_{n \times d} \right), \quad (32)$$

where the mapping $\phi(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^p$ is applied position-wise. Unfortunately, the feature map of the exponential kernel is infinite dimensional. Hence, any finite kernel is an approximation of the attention matrix and may be interpreted as a low-rank factorization. However, they are presented separately here due to their conceptual difference. Katharopoulos et al. [56] approximated the attention matrix in the Linear Transformer with the feature map $\phi(x) = \text{elu}(x) + 1$, where the function $\text{elu}(\cdot)$ denotes the exponential linear unit given by:

$$\text{elu}(x) = \begin{cases} \alpha(e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}, \quad (33)$$

where α is a hyperparameter. The Linear Transformer performed on par with the vanilla Transformer on autoregressive image generation, but poorly on automatic speech recognition.

Choromanski et al. [18] later demonstrated that the exponential is equivalent to a kernel with a randomized mapping:

$$\exp(x^\top y) = \mathbb{E}_{w \sim \mathcal{N}(0, I_d)} \left[\exp \left(w^\top x \frac{\|x\|^2}{2} \right) \exp \left(w^\top y \frac{\|y\|^2}{2} \right) \right]. \quad (34)$$

Consequently, the authors introduced the Performer, a linear complexity model that approximates the attention by means of a kernel with the following feature mapping:

$$\phi(x) = \frac{\exp(-\|x\|^2/2)}{\sqrt{2p}} \left[\exp(w_1^\top x); \dots; \exp(w_p^\top x); \exp(-w_1^\top x); \dots; \exp(-w_p^\top x) \right], \quad (35)$$

where $w_i \sim \mathcal{N}(0, I_d)$. To further reduce the variance of the estimator, w_i are constrained to be exactly orthogonal, which is achieved with the Gram-Schmidt process. The hyperparameter p corresponds to the number of random features and controls the quality of the approximation.

Clustering and Locality-sensitive Hashing [122]: As previously explained, clustering can uncover sparse patterns by grouping queries and keys and only computing the attention between positions within the same cluster. Alternatively, Vyas et al. [122] proposed to factorize the attention with clustering by grouping queries into a fixed number of non-overlapping clusters and by computing the attention between the cluster's centroids and the keys. Consequently, the attention score is only computed once per group of similar queries and broadcasted to all, resulting in linear complexity. Since queries may be clustered differently across attention heads and, since the attention sub-layer includes a residual connection, two queries in the same cluster can have different output representations. The authors proved that the approximation error for a given query is bounded by its distance to its centroid multiplied by the spectral norm of the keys matrix. As such, the K-Means algorithm can be used for minimizing the approximation error. However, K-Means in the original space would be slow to compute, as Lloyd algorithm has a complexity of $O(ncl)$, where c is the number of clusters and l is the number of Lloyd iterations. Instead, the authors first used a locality-sensitive hashing scheme on the queries before applying K-Means with the Hamming distance, which reduces the complexity to $O(ncl + cbl + ndb)$, where b is the number of bits used for hashing.

To further improve the approximation, Vyas et al. [122] proposed the *improved cluster attention* that separately consider the k keys with the highest attention for each cluster. Intuitively, keys with high approximated attention may have low attention for some queries, resulting in a large approximation error. As a solution, the dot product between these top- k keys and all queries belonging to the corresponding cluster is computed. Then, the attention is rescaled by the total probability mass assigned to these top- k keys.

Compared to the Reformer, Vyas et al. [122] method is significantly faster (43% lower epoch time) while being significantly more accurate (35% lower phone error rate) for speech recognition on the *Wall Street Journal*.

4.3 Architectural Change

Finally, the Transformer's complexity may also be reduced by modifying the model's architecture and preserving the original attention mechanism. Let us investigate (i) the Transformer-XL and the Compressive Transformer that rely on memory and (ii) then the Funnel-Transformer that iteratively compresses sequences.

Memory [22, 93]: The block-wise approach splits the input sequence into small non-overlapping subsequences called windows, blocks, or chunks, which are processed independently; therefore, the maximum dependency length is equal to that of the subsequence. To leverage

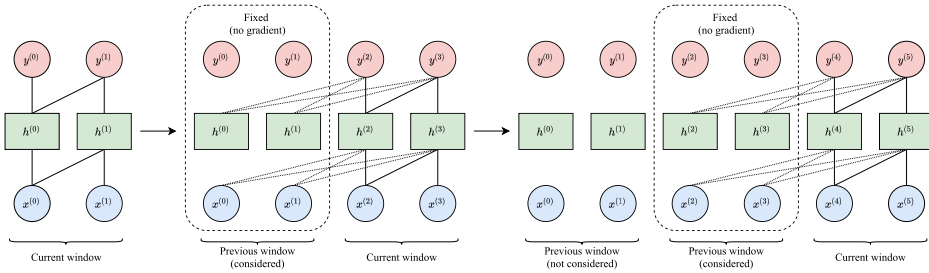


Fig. 18. Segment-based recurrence, which is similar to truncated BPTT. The window size is equal to two, and only the previous window is considered. For the sake of clarity, parameters from and to states that do not contribute are omitted.

information from previous windows, Dai et al. [22] introduced the Transformer-XL, which relies on segment-based recurrence between windows. This recurrence mechanism is implemented by storing the representations of the previous window in a **first-in first-out memory (FIFO)**. Then, the attention mechanism can attend to the representations located in this memory, but the gradients are not computed for the attention on these elements. Although this model achieves a RECL four times greater than the vanilla Transformer with the same parameter budget, it cannot capture dependencies outside the FIFO memory range. Furthermore, this model is only compatible with autoregressive tasks. This technique is analogous to truncated **backpropagation through time (BPTT)**, except that a sequence of hidden states is considered instead of the previous one. Figure 18 illustrates the segment-based recurrence of the Transformer-XL.

To further increase the range of dependencies considered by the Transformer-XL, Rae et al. [93] proposed the Compressive Transformer, which adds a compressed memory to the original FIFO memory. Representations of past windows are first stored in the standard FIFO memory, like the Transformer-XL. Then, when this memory is full, the oldest representations are compressed with a user-defined function and stored in the compressed FIFO memory instead of being discarded. The number of elements considered in the original FIFO memory to generate the compressed memory depends on the chosen function. The authors propose using max/mean pooling, 1D convolution, dilated convolutions, or the most attended representations by the attention. They also proposed to learn the compression function with an auxiliary auto-encoding loss and a variant called attention-reconstruction loss, which typically reconstructs the original memory from the compressed ones. They show a clear advantage over the Transformer-XL on NLP tasks and comparable results on speech modelling.

Sequence Compression [21]: Many tasks such as image classification and sentiment analysis only require producing a single output for the whole sequence. Dai et al. [21] argued that the full-length sequence of hidden states may contain significant redundancy and that the model may not have to preserve token-level information. Consequently, they proposed the Funnel-Transformer, whose encoder reduces the computational cost by gradually reducing the length of the hidden states sequence with pooling. Note that instead of directly feeding the pooled sequence into the attention layer, it is only used to construct the query matrix, while the unpooled sequence is used to construct the key and value matrices. Additionally, the authors proposed to recover the original sequence length by up-sampling the compressed sequence of hidden states to address the common pre-training objectives, such as MLM, that require separate representation for each token. Although the Funnel-Transformer effectively reduces the computational and memory cost of the encoder, the complexity remains quadratic, and the best performances are achieved on tasks that only require sequence-level representation.

5 SHORTCOMINGS

This section discusses the lack of understanding of the self-attention inner workings and the limitation of the Transformer evaluation methodology, including the lack of standard benchmarks for long-range dependencies.

Self-attention is a relatively new mechanism that has been quickly and widely adopted due to its remarkable empirical success. Nonetheless, the self-attention inner workings are not yet fully understood, and many questions remain unanswered, including why it works, what it learns, and whether it is interpretable. Answering those questions is crucial to designing faster and lighter Transformers that are competitive with the original model. As of this article's writing, the deep learning community actively investigates self-attention and has proposed preliminary answers to the aforementioned questions. For instance, evidence supporting both the self-attention interpretability [101, 131] and non-interpretability [53] have been published. Tay et al. [113] empirically evaluated the dot product impact on natural language processing tasks and concluded that query-keys interaction is *"useful but not that important."* Kitaev et al. [59] investigated the impact of sharing queries and keys and concluded that *"it turns out that sharing QK does not affect the performance of Transformer."*

Despite our current limited understanding of the self-attention mechanism, a wide range of faster and lighter Transformers has been introduced in a short amount of time, each claiming comparable or superior performance to the vanilla Transformer. Since there is no consensus on how to evaluate the proposed approaches [115], researchers often have to evaluate their method on a small range of tasks. However, different tasks may require different assumptions, which means that one method may work well on a specific task but poorly on others. For instance, Tay et al. [113] showed that a simple Synthesizer is highly competitive with the vanilla Transformer across a range of natural language processing tasks, including machine translation, language modelling, and text generation. However, Tay et al. [115] later showed that the vanilla Transformer outperforms the Synthesizer on the more difficult Long-Range Arena benchmark. Long-Range Arena [115] is a suite of five general and challenging tasks designed to evaluate how well Transformers capture long-term dependencies from different modalities such as text, natural and synthetic images, and mathematical expressions. Table 3 compiles the Long-Range Arena results of the models discussed in the survey. For a complete description of the objectives and datasets, we refer the reader to the original paper.

Furthermore, due to Transformers' large training cost, researchers often evaluate their approach against a limited number of models on the tasks of interest. For instance, [59] only evaluated the Reformer against three distinct vanilla Transformers [85, 121] on three tasks. Standardized suites of benchmarks such as GLUE and the recent Long-Range Arena allow researchers and practitioners to evaluate only their method and compare it against a public leaderboard. Consequently, we highly recommend that researchers consider such benchmarks.

Although standardized benchmarks such as Long-Range Arena would help compare the models, the results should be taken with caution, since the performance depends on the model size and hyperparameters, the speed depends on the implementation and hardware, and the memory footprint depends on the implementation and general methods used. For instance, the Switch Transformer uses a mixture of experts, mixed-precision, expert dropout, knowledge distillation, and a careful initialization. Therefore, it is difficult to isolate the benefit of a single modification.

Finally, the complexity is not always representative of the practical efficiency. For instance, the Reformer achieves an asymptotic complexity of $O(n \log n)$ but is significantly slower than the vanilla Transformer on small sequences, as shown in Table 3. This slowdown is due to large constants hidden in the complexity. Even when there are no hidden constants, there is a distinction between theoretical complexity and what is achievable in practice. For instance, sparse matrix multiplication may reduce the complexity from quadratic to linear in theory. However, it is well

Table 3. Long-Range Arena benchmark [115]

Models	Average score (%)	Steps per second		Peak memory (GB)	
		1k	4k	1k	4k
Transformer [121]	54.39	8.1	1.4	0.85	9.48
Sparse Transformer ¹⁰ [16]	51.24				
Longformer ¹⁰ [5]	53.46				
BigBird [139]	55.01	7.4	1.5	0.77	2.88
Sinkhorn Transformer [114]	51.39	9.1	5.3	0.47	1.48
Reformer [59]	50.67	4.4	1.1	0.48	2.28
Linformer [126]	51.36	9.3	7.7	0.37	0.99
Synthesizer [113]	51.39	8.7	1.9	0.65	6.99
Linear Transformer [56]	50.55	9.1	7.8	0.37	1.03
Performer [18]	51.41	9.5	8.0	0.37	1.06

Results have been compiled from the original paper. Benchmarks are run on 4×4 TPU V3 chips, and the memory is reported per device.

known that GPUs and TPUs are not designed to perform such operations efficiently [11] and, in practice, sparse matrix multiplication is often slower than dense ones. We encourage researchers to explicitly report the complexity as well as the number of **floating operations (FLOPs)**, the wall-clock time with the hardware, and the memory footprint of their method.

6 BROADER IMPACT OF EFFICIENT TRANSFORMER

This section extends the three motivations and potential impacts of lighter and faster Transformers briefly discussed in Section 2.4.

First and foremost, computational resources are not only finite but also expensive. Consequently, there are severe inequalities between research groups and between companies. Indeed, many researchers do not have access to GPU or TPU farms, and most companies cannot afford to spend thousands or millions of dollars on dedicated hardware, especially if deep learning is not their primary focus. At this time, the resources disparities have increased dramatically to a point where only a few parties can afford to train massive state-of-the-art models. A prime example of this cleavage is the Transformer. Indeed, the largest Transformers are so expensive to train, even for large companies such as Microsoft, that they are only trained once. For instance, Brown et al. [10] noticed an issue in their pre-processing after training GPT-3. As the author explained, they could not train their model again due to the massive cost and therefore published their results with a known issue. Resources inequalities also hinder creativity, as researchers with promising ideas may not be able to implement them, thus reinforcing the vicious “rich get richer” circle, where well-funded groups and companies that have access to more resources are more likely to achieve state-of-the-art results and receive more fundings [108].

Additionally, lower-complexity Transformers enable novel applications, as extremely long sequences cannot be processed in a reasonable amount of time by the quadratic complexity vanilla Transformer. For instance, Choromanski et al. [18] observed the Performer’s potential impact on biology, and Zaheer et al. [139] evaluated BigBird on genomics tasks that take fragments of DNA as input. Huang et al. [46] were able to generate minute-long musical compositions with a Transformer that leverages the block-wise approach and an efficient computation of the relative attention. Note that, contrary to the attention introduced by Reference [121], the relative attention [102]

¹⁰The Sparse Transformer and Longformer depend on CUDA kernels that are difficult to implement on TPUs. Therefore, Tay et al. [115] used *equivalent* implementations to emulate their performance and did not report their efficiency.

explicitly models the input positions. The range of applications will surely expand as researchers design ever-lighter and -faster Transformers.

Finally, recent research made it clear that we must cut carbon dioxide (CO₂) emissions in half over the next decade to limit global warming. The large-scale infrastructures used by the deep learning community consume a considerable amount of electricity, which is mainly produced by non-renewable sources such as coal or gas [49]. Strubell et al. [108] estimated that training a Transformer with neural architecture search generates up to 284,000 kg of CO₂. For reference, the average American emits 16,400 kg of CO₂ per year, and the average car emits about 57,200 kg during its lifetime¹¹ (fuel included). The authors estimated that training a single instance of BERT [24] on GPU produces about the same amount of CO₂ as a trans-American flight. Although lighter and faster models require fewer resources and therefore produce less carbon dioxide, they are also more accessible, so we would expect more models to be trained. Overall, it is difficult to know whether lighter and faster Transformers will positively impact the environment. Nonetheless, researchers and practitioners ought to have in mind the significant environmental impact of their experiments, which can be estimated with the Machine Learning Emissions Calculator¹² developed by Luccioni et al. [75].

7 FUTURE RESEARCH DIRECTIONS

In our opinion, the current research directions follow one of two purposes: (i) efficiency and affordability or (ii) generalization performance. Since this survey addresses approaches to yield faster and lighter Transformers, let us start with the efficiency and affordability objective.

7.1 Efficiency and Affordability

To the best of our knowledge, researchers and practitioners have not yet identified a specialized approach that improves the Transformer's efficiency for every task, dataset, and hardware, as explained in Section 5. In our opinion, one of the most promising avenues is to learn adaptively sparse patterns that are structured for the available hardware. Let us justify our claim.

The Softmax function only contains a few large values due to its exponential nature. Therefore, it can be effectively approximated by masking the positions with small weights. In theory, the computation and memory reduction is linearly proportional to the ratio of masked positions. In practice, however, the improvement depends on the hardware. As of this survey's writing, NVIDIA is the first and only manufacturer to offer an architecture that natively supports sparse operations, resulting in a virtually perfect speedup. One may reasonably expect other manufacturers to follow this direction due to the prevalence of sparse operations in deep learning. Therefore, the sparse patterns should be structured such that the hardware natively supports them. Handcrafting features or patterns based on prior knowledge is known to be suboptimal. Instead, the model should learn the patterns from the data for the task at hand. Additionally, individual samples are likely to require different attention patterns, and hence, the patterns should be adaptative (content-based). Finally, we believe it is beneficial to include global tokens, since they allow any position to attend to any other position in two layers, thus preserving the attention's expressiveness.

7.2 Generalization Performance

A second research venue consists in improving the network generalization performance. Since the deep learning renaissance associated with greedy layer-wise unsupervised pre-training [36], there has been a clear trend towards scaling up neural networks. As a result, researchers and

¹¹A product lifetime or lifecycle typically includes material production, manufacturing, usage, and end-of-life disposal.

¹²<https://mlco2.github.io/impact/>.

practitioners have been able to leverage ever-larger datasets and ultimately improve the network's performance. In this setting, scaling is performed typically by increasing the number of layers, the number of attention heads, the input embedding dimension, and the feedforward network width.

Among others, Radford et al. [92] introduced a large Transformer called GPT-2 and evaluated various model sizes on language modelling tasks in a zero-shot setting. The authors reported that the performance significantly increased with the model size ranging from 117M to 1.5B parameters. Recently, Brown et al. [10] introduced GPT-3 based on the GPT-2 architecture and considered an even wider span of model sizes, ranging from 125M to 175B parameters. The authors reported that the model performance smoothly increased with the model size in most cases and suggested that this trend should extend to even larger models. Furthermore, Devlin et al. [24] investigated the effect of BERT size on the GLUE benchmark and concluded that *"larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks."*

These observations suggest that researchers and practitioners must scale their model to pursue the generalization performance objective. Inherently, scaling is resource-expensive and goes against the affordability sought in this survey. Nonetheless, there are research directions to improve the generalization capability of deep learning models that are orthogonal to scaling and thus compatible with efficiency. A promising avenue is structural inductive biases. A recent structural inductive bias inspired by independent mechanisms in the causality literature consists of designing an architecture that learns sparsely interacting modules, each one of them specialized in a different mechanism [37]. Ideally, individual modules should be robust to changes in the aspects of the world that are unrelated to this module, such as in the case of distributional shift. Lamb et al. [61] applied this idea to Transformers by introducing the **Transformers with Independent Mechanisms (TIM)**. The authors observed that TIM layers could be combined with the mixture of experts approach, allowing the switching to be specific to distinct aspects of the data.

Combining universally effective and efficient approaches such as the aforementioned sparse patterns with conditional computing and the independent mechanisms prior appears to be promising to tackle complex tasks without relying on large-scale resources.

8 CONCLUSION

Transformers have quickly become the de facto model for processing sequences, notably achieving state-of-the-art in most natural language processing tasks at the cost of quadratic complexity. As a result, researchers have leveraged numerous techniques to mitigate this memory and computational burden. This survey investigated popular general methods to make neural networks lighter and faster and discussed their strengths and limitations. Notably, we advised researchers and practitioners to use mixed-precision and gradient checkpointing due to their simplicity and overall benefits. Often, these general techniques are not sufficient to mitigate the Transformer's complexity. Consequently, this survey reviewed the lower-complexity variations of the Transformer and discussed their assumptions, justification, and shortcomings. Notably, we advised researchers and practitioners to rely on pre-trained models whenever possible. Otherwise, we recommend training a small vanilla Transformer with mixed-precision and gradient checkpointing to apprehend the dependencies required for the task and select the appropriate models accordingly. Additionally, we discussed the potential impacts of affordable Transformers, including improving the state-of-the-art, extending the range of applications, increasing the equity between researchers, and potentially reducing the environmental impact. Finally, we highlighted promising future research directions for this exciting architecture.

APPENDICES

A INTRODUCTION TO MACHINE LEARNING

At the dawn of **artificial intelligence (AI)**, researchers rapidly tackled and solved problems that were challenging for humans but relatively straightforward for computers in that they could be described as a set of rules. Chess may certainly be the epitome of such complex tasks solved brilliantly by artificial intelligence. Nonetheless, despite the achievements of AI, simpler tasks that humans solve instinctively proved to be much more challenging, as they are not easily expressed formally. Among others, speech and object recognition were—and still are, to some extent—challenging problems to solve for computers. **Machine learning (ML)** provides a solution to intuitive problems by allowing computers to learn from experience instead of relying on human knowledge to specify the tasks or their solution. The seemingly ever-increasing amount of data produced every day has enabled machine learning to become suitable and successful for a wide range of simple and complex problems. Since the renaissance of **deep learning (DL)** associated with greedy layer-wise pre-training, neural networks have become the most popular family of algorithms for machine learning as they learn a hierarchy of concepts, with each concept defined through its relation to simpler concepts. As of the writing of this survey, deep learning, and more generally artificial intelligence, has become a thriving field with numerous practical applications that directly impact countless human lives, from medical diagnoses to movie recommendations.

B PRACTICAL GUIDELINES - GENERAL METHODS

The general approaches presented in Section 3 apply to the original Transformer as well as its lower-complexity alternatives. Therefore, they are discussed before introducing the specialized approaches in Section 4. In particular, this section provides practitioners and researchers with a series of guidelines on which methods to apply, depending on the bottleneck and whether it occurs during optimization or inference. The distinction between optimization (i.e., pre-training and training) and inference is motivated by the former being significantly more resource-intensive than the latter.

The primary focus of this survey is to make Transformers more efficient and ultimately more affordable. Therefore, only substantial performance losses will be mentioned, along with other significant drawbacks such as incompatibilities and instabilities. Unless specified otherwise, the methods are readily available in PyTorch [87] and Tensorflow [1], two standard deep-learning libraries.

B.1 Optimization

Optimization is the most resource-intensive phase, prominently due to the iterative nature of the process, the quadratic complexity of the attention mechanism, and the in-memory recording of intermediate values during the forward pass. Consequently, most of the above approaches to reduce computation, memory, or both, focus on optimization.

B.1.1 Computation Savings. Recently, the undeniable success of pre-trained Transformers such as BERT [24], ViT [27], and GPT-3 [10] has confirmed the benefits of unsupervised pre-training. As previously mentioned, pre-training initializes the network's weights in a “good” region of space that allows the model to converge faster. Therefore, we advise practitioners and researchers to build upon pre-trained models like the ones available on the open-source library Hugging Face [132].

Nonetheless, pre-trained models are typically only available for “conventional” data and tasks such as translation, summarization, question answering, text-to-speech, image classification, and object detection. As for data and tasks without pre-trained models, we recommend initializing the

model with a principled strategy such as Admin or T-Fixup and using a sample-efficient objective. Those techniques are not yet implemented in standard libraries, therefore, we suggest using T-Fixup, as it is simpler than Admin.

B.1.2 Memory Savings. As discussed before, although time may limit one's experiments, memory bottlenecks are much more critical. Since the intermediates values are responsible for a substantial part of the memory footprint, the first method to apply whenever memory is the main limiting factor during optimization is gradient checkpointing. The approach has two significant advantages: (i) the tradeoff between memory and computation controlled by the number of intermediate values kept in memory is highly adjustable, and (ii) the method is straightforward to use in TensorFlow¹³ and PyTorch.¹⁴ Nevertheless, gradient checkpointing has some caveats with multiple GPUs, even on a single machine. For instance, as of the writing of this survey, gradient checkpointing interferes with PyTorch's Distributed and Data Parallel API, leading to instabilities.¹⁵

Alternatively to gradient checkpointing, reversible layers provide a mechanism to recompute the intermediate values during the backward pass, thereby decoupling the model's depth from the amount of memory required by the activations. Although the increase in computation is reasonable, reversible layers produce numerical errors that may accumulate layers after layers to the point that they become an issue. Additionally, reversible layers are not yet part of standard libraries and require manually writing the forward and backward operations.

In addition to gradient checkpointing or reversible layers, parameter sharing allows further reducing the memory and is straightforward. However, unlike the other approaches, parameter sharing reduces the model's capacity. Fortunately, the tradeoff between capacity and memory/computation savings is highly customizable, depending on the number of parameters shared.

Finally, a mixture of experts potentially with micro batching is expected to allow many memory-limited GPUs to train a Transformer even if each GPU is individually too small. However, both approaches require substantial effort to implement and impose a communication cost.

B.2 Inference

Sometimes, researchers have the resources to train large models during the development phase due to public or academic infrastructures, but they do not have the resources to deploy them. In such cases, one may do a neural architecture search to find the best model within a parameter budget during training, preferably with So et al. [105]'s approach. As of this survey's writing, neural architecture search is not part of standard libraries.

Alternatively or additionally to NAS, structured pruning and distillation reduce the amount of memory and computations with fine-grained control. While structured pruning is already implemented, distillation is as easy as building a second model that predicts the teacher's output. As the aforementioned results suggest [54, 79, 98, 99, 120], the Transformer's performance does not significantly degrade when the model is pruned or distilled. Therefore, to reduce the amount of energy consumed by the model, we suggest applying those methods even when resources are sufficient during inference.

B.3 Optimization and Inference

The first and foremost method for faster and lighter models is automatic mixed-precision. Mixed-precision is compatible with virtually every neural network, combines with every other approach,

¹³<https://github.com/cybertronai/gradient-checkpointing>.

¹⁴<https://pytorch.org/docs/stable/checkpoint.html>.

¹⁵<https://discuss.pytorch.org/t/ddp-and-gradient-checkpointing/132244/2>.

reduces the memory footprint, and accelerates computations on modern GPUs. Additionally, this method is one of the simplest to implement, only requiring a few lines of code in PyTorch¹⁶ and TensorFlow.¹⁷

Although 8-bit quantization may seem similar to 16-bit mixed-precision, the former is primarily used to speed up inference and is not as readily available as the latter. In particular, PyTorch does not provide quantized operators for GPU as of the writing of this survey, and Tensorflow warns users that “*different hardware may have preferences and restrictions that may cause slight deviations when implementing the spec that result in implementations that are not bit-exact.*”¹⁸ Due to the finicky nature of 8-bit quantization, we suggest reserving this approach to specific hardware and use-cases such as the mobile setting.

C PRACTICAL GUIDELINES - SPECIALIZED METHODS

With limitations mentioned in Section 5 in mind, let us examine the results of Reference [115] and draw some broad guidelines.

The first observation is that every model is lighter than the original Transformer. Nonetheless, for memory-limited environments, the Synthesizer is the least advisable alternative, as the model only reduces the memory by 24% to 26%, regardless of the sequence length, which is consistent with its quadratic complexity. Instead, the Linformer, Performer, and Linear Transformer are better suited to address memory bottlenecks, as they are at least 56% and 88% lighter than the original Transformer for input sequences of 1,000 and 4,000 tokens, respectively, which is also consistent with their linear complexity.

The second observation is that, on TPU V3 chips, the Synthesizer, the Reformer, and BigBird perform roughly the same number of steps per second as the original Transformer, regardless of the sequence length. In contrast, the Linformer, Performer, Sinkhorn Transformer, and Linear Transformer are significantly faster than the original Transformer for input sequences of 4,000 tokens while performing on par for sequences of 1,000 tokens. Consequently, those models are better suited for computation-limited environments. We do not wish to overstate our claims here, since TPUs and GPUs differ on some key aspects,¹⁹ and speedups may significantly vary, as observed by Wang et al. [128] and Wang et al. [127]. Although the data processing pipeline and the model implementation are outside this survey’s scope, they should be tuned for the exact hardware used, as it may significantly impact the performance.

Nonetheless, it would seem that the Linformer, Performer, and Linear Transformer are excellent options to improve memory and computation, with the Linformer standing out, considering the simplicity of its implementation. However, those models also have serious drawbacks. The Linformer requires instantiating the projection matrices E and F of dimension $k \times n$, and thus can only process fixed-sized input sequences. Therefore, sequences must be padded to the size of the largest one in the dataset, which may significantly degrade the model’s efficiency. The Performer and Linear Transformer are challenging to be efficiently implemented. Besides, they perform noticeably worse than the original Transformer, on average. In some cases, such as byte-level text classification, they manage to outperform the original Transformer. In other cases, however, they might critically underperform. For instance, in a longer variant of the ListOps task [81] that consist of modelling hierarchically structured data, they achieve less than 50% of the original Transformer’s performance.

¹⁶https://pytorch.org/tutorials/recipes/recipes/amp_recipe.html.

¹⁷https://www.tensorflow.org/guide/mixed_precision.

¹⁸https://www.tensorflow.org/lite/performance/quantization_spec#specification_summary.

¹⁹ Compared to modern GPUs with Tensor Cores, TPUs typically perform more FLOPs but have a lower memory bandwidth, have fewer but larger tiles and apply the activation function within the matrix multiplication.

In contrast, sparse Transformers suffer less performance degradation, on average, as measured on the Long-Range Arena benchmark. Notably, the LongFormer and BigBird achieved the same accuracy as the original Transformer for the ListOps task. Sparse models have, however, two major shortcomings. First, the sparsity must be structured to be efficiently implemented and yield practical improvements. Otherwise, the sparse model may be slower than its dense equivalent. Furthermore, CUDA kernels require considerable effort to be efficiently implemented and are specific to GPUs. Implementing equivalent kernels on TPUs is challenging, or even impossible, due to the disparity in supported primitives and operations. Second, dependencies that must be modelled to solve the task accurately should not be masked. Otherwise, the performance will be critically impacted. To select the appropriate sparse model, we recommend that one train a small vanilla Transformer with mixed-precision and gradient checkpointing and then analyze the activation patterns of each layer’s attention.

Nonetheless, in a recent paper, Narang et al. [82] investigated the impact of numerous modifications to the Transformer architecture, including changes in activation, normalization, depth, embeddings, and Softmax, on three NLP benchmarks, namely, SuperGLUE [123], XSum [83], and WebQ [8]. The authors also evaluated several methods studied in this article, including parameter sharing, Synthesizers, the Switch Transformer, and the Universal Transformer. They observed that no modification was able to improve the performance significantly. After ruling out various explanations, the authors conjectured that “*modifications to the Transformer architecture often do not transfer across implementations and applications*,” which may explain why no modification has been widely adopted.

In conclusion, there seem to be no simple and universal guidelines regarding the current Transformer alternatives. If the data and task are standard, then we recommend looking in the literature or on the [Papers With Code](#) website for references on how the different methods compare and experiment with already pre-trained models. Otherwise, we recommend using a small vanilla Transformer with mixed-precision and gradient checkpointing as baseline, then experimenting with already-implemented lower-complexity models. As a side note, one may also want to combine multiple specialized approaches. For instance, BigBird-ETC relies on additional tokens for global attention, a form of memory similar to the Compressive Transformer. Nonetheless, many combinations are unprincipled at best. For instance, one should not factorize a sparse attention: The complexity will be similar to that of the same factorization of the full attention, and the sparsity may lose valuable information that the factorization could have preserved.

D ALTERNATIVES TO SELF-ATTENTION

Recently, attention-free alternatives to the Transformer have been proposed, putting Vaswani et al.’s [121] original paper title *Attention Is All You Need* to the test. Such architectures have not been explored in the core of this survey, as they arguably remove the Transformer’s core mechanism. Nonetheless, it is important to mention some of the most popular and promising alternatives.

Tolstikhin et al. [118] argued that self-attention is not required for image classification. They introduced a model called MLP-Mixer solely based on a succession of two multilayer perceptrons applied independently to image patches and channels, respectively, which achieved comparable accuracy to the ViT [27] on ImageNet.

Likewise, Liu et al. [70] argued that self-attention is not critical for computer vision and language modelling. They introduced a network called gMLP that models the interactions with **Spatial Gating Units (SGU)** instead of self-attention. Their model achieved the same accuracy as the ViT [27] on ImageNet, and the same perplexity of BERT [24] on a subset of C4.

Alternatively, Bello [4] proposed to replace the Transformer’s self-attention with Lambda layers. Long-range content and position-based interactions are captured by transforming the context into

linear functions, i.e., matrices, and applying them to each input independently. LambdaNetworks achieved comparable results to relatively small Transformers on ImageNet classification. While the memory complexity of Lambda layers remains quadratic with respect to the sequence length, it does not scale with the batch size. Additionally, the author proposed a multi-query variant that scales down the complexity by a factor.

Finally, Yu et al. [137] argued that the architecture of the Transformer is more valuable to the performance than the specific mechanism to relate the tokens. To illustrate their claim, the authors introduced the PoolFormer, a network that performs similarly to vision Transformers while replacing the self-attention mechanism with pooling, a simple non-parametric operator. Furthermore, the authors expanded on this idea with a more general and flexible architecture called MetaFormer, where the mechanism to relate the tokens is not specified while the other components are kept the same as the Transformer.

E SUMMARY OF THE SPECIALIZED APPROACHES

Table 4. Summary of the Specialized Methods and Their Associated Models

Category	Approach	Model
Sparse	Fixed and Random Patterns	Star-Transformer [41]
		Sparse Transformer [16]
		Cascade Transformer [125]
		LogSparse-Transformer [66]
		BlockBERT [90]
		Longformer [5]
	Learned and Adaptive Patterns	BigBird [139]
		Sinkhorn Transformer [114]
		SparseBERT [104]
		Adaptively Sparse Transformer [20]
Factorized Attention	Clustering and Locality-sensitive Hashing	Reformer [59]
		Routing Transformer [96]
	Low-rank Factorization	Linformer [126]
		Synthesizers [113]
		Nyströmformer [136]
	Kernel Attention	Linear Transformer [56]
		Performer [18]
	Clustering and Locality-sensitive Hashing	Transformer with clustered attention [122]
Architectural Change	Memory	Transformer-X Dai et al. [22]
		Compressive Transformer Rae et al. [93]
	Sequence Compression	Funnel-Transformer Dai et al. [21]

REFERENCES

- [1] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, et al. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Retrieved from <https://www.tensorflow.org/>.
- [2] Jimmy Ba and Rich Caruana. 2014. Do deep nets really need to be deep? In *NIPS*, Vol. 27.
- [3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *ICLR*.
- [4] Irwan Bello. 2021. LambdaNetworks: Modeling long-range interactions without attention. In *ICLR*.
- [5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv e-prints* (2020), arXiv:2004.05150.

- [6] Yoshua Bengio. 2013. Deep learning of representations: Looking forward. In *SLSP*, Vol. 7978, 1–37.
- [7] Yoshua Bengio. 2013. Estimating or propagating gradients through stochastic neurons. *CoRR* abs/1305.2982 (2013).
- [8] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on freebase from question-answer pairs. In *EMNLP*. 1533–1544.
- [9] Ondrej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, et al. 2014. Findings of the 2014 workshop on statistical machine translation. In *SIGMT*. 12–58.
- [10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, et al. 2020. Language models are few-shot learners. In *NeurIPS*, Vol. 33, 1877–1901.
- [11] A. Buluc and J. R. Gilbert. 2008. Challenges and advances in parallel sparse matrix-matrix multiplication. In *ICPP*. 503–510.
- [12] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-end object detection with transformers. In *ECCV*. 213–229.
- [13] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. 2016. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *ICASSP*. 4960–4964.
- [14] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *CoRR* abs/1604.06174 (2016).
- [15] Jianpeng Cheng, Li Dong, and Mirella Lapata. 2016. Long short-term memory-networks for machine reading. In *EMNLP*. 551–561.
- [16] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *CoRR* abs/1904.10509 (2019).
- [17] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, et al. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *EMNLP*. 1724–1734.
- [18] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, et al. 2021. Rethinking attention with performers. In *ICLR*.
- [19] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *ICLR*.
- [20] Gonalo M. Correia, Vlad Niculae, and Andr  F. T. Martins. 2019. Adaptively sparse transformers. In *EMNLP-IJCNLP*. 2174–2184.
- [21] Zihang Dai, Guokun Lai, Yiming Yang, and Quoc Le. 2020. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. In *NeurIPS*.
- [22] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In *ACL*. 2978–2988.
- [23] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. 2019. Universal transformers. In *ICLR*.
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*. 4171–4186.
- [25] Laurent Dinh, David Krueger, and Yoshua Bengio. 2015. NICE: Non-linear independent components estimation. In *ICLR*.
- [26] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. 2017. Density estimation using real NVP. In *ICLR*.
- [27] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, et al. 2021. An image is worth 16 × 16 words: Transformers for image recognition at scale. In *ICLR*.
- [28] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-adaptive transformer. In *ICLR*.
- [29] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *J. Mach. Learn. Res.* 20, 55 (2019), 1–21.
- [30] William Fedus, Barret Zoph, and Noam Shazeer. 2021. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv e-prints* (2021), arXiv:2101.03961.
- [31] Quentin Fournier, Daniel Alose, Seyed Vahid Azhari, and Franois Tetreault. 2021. On improving deep learning trace analysis with system call arguments. In *MSR*. 120–130.
- [32] Jonathan Frankle and Michael Carbin. 2019. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*.
- [33] Andrea Galassi, Marco Lippi, and Paolo Torroni. 2021. Attention in natural language processing. *IEEE Transactions on Neural Networks and Learning Systems* 32, 10 (2021), 4291–4308. DOI : [10.1109/TNNLS.2020.3019893](https://doi.org/10.1109/TNNLS.2020.3019893)
- [34] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, Vol. 9, 249–256.
- [35] Aidan N. Gomez, Mengye Ren, Raquel Urtasun, and Roger B. Grosse. 2017. The reversible residual network: Back-propagation without storing activations. In *NeurIPS*, Vol. 30.

- [36] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. Retrieved from <http://www.deeplearningbook.org>.
- [37] Anirudh Goyal, Alex Lamb, Jordan Hoffmann, Shagun Sodhani, Sergey Levine, Yoshua Bengio, et al. 2019. Recurrent independent mechanisms. *CoRR* abs/1909.10893 (2019).
- [38] Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *CoRR* abs/1603.08983 (2016).
- [39] Scott Gray, Alec Radford, and Diederik P. Kingma. 2017. GPU kernels for block-sparse weights. <https://cdn.openai.com/blocksparse/blocksparspaper.pdf>.
- [40] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, et al. 2020. Conformer: Convolution-augmented transformer for speech recognition. In *Interspeech*. 5036–5040.
- [41] Qipeng Guo, Xipeng Qiu, Pengfei Liu, Yunfan Shao, Xiangyang Xue, and Zheng Zhang. 2019. Star-transformer. In *NAACL-HLT*. 1315–1325.
- [42] K. He, X. Zhang, S. Ren, and J. Sun. 2016. Deep residual learning for image recognition. In *CVPR*. 770–778.
- [43] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv e-prints* (2015), arXiv:1503.02531.
- [44] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computat.* 9, 8 (1997), 1735–1780.
- [45] Sara Hooker. 2020. The hardware lottery. *CoRR* abs/2009.06489 (2020).
- [46] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, et al. 2019. Music transformer. In *ICLR*.
- [47] Xiao Shi Huang, Felipe Pérez, Jimmy Ba, and Maksims Volkovs. 2020. Improving transformer optimization through better initialization. In *ICML*, Vol. 119, 4475–4483.
- [48] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, et al. 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *NeurIPS*, Vol. 32.
- [49] IEA. 2018. World gross electricity production, by source, 2018. Retrieved from <https://www.iea.org/data-and-statistics/charts/world-gross-electricity-production-by-source-2018>.
- [50] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, Vol. 37, 448–456.
- [51] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF*. 2704–2713.
- [52] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computat.* 3 (1991), 79–87.
- [53] Sarthak Jain and Byron C. Wallace. 2019. Attention is not explanation. *CoRR* abs/1902.10186 (2019).
- [54] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, et al. 2020. TinyBERT: Distilling BERT for natural language understanding. In *EMNLP*. 4163–4174.
- [55] Shigeki Karita, Nanxin Chen, Tomoki Hayashi, Takaaki Hori, Hirofumi Inaguma, Ziyang Jiang, et al. 2019. A comparative study on transformer vs RNN in speech applications. In *ASRU*. 449–456. DOI: <https://doi.org/10.1109/ASRU46091.2019.9003750>
- [56] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, Vol. 119, 5156–5165.
- [57] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. 2022. Transformers in vision: A survey. *ACM Comput. Surv.* 54, 10s (2022). <https://doi.org/10.1145/3505244>
- [58] Urvashi Khandelwal, He He, Peng Qi, and Dan Jurafsky. 2018. Sharp nearby, fuzzy far away: How neural language models use context. In *ACL*. 284–294.
- [59] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. In *ICLR*.
- [60] Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, et al. 2020. Big transfer (BiT): General visual representation learning. In *ECCV*, Vol. 12350, 491–507.
- [61] Alex Lamb, Di He, Anirudh Goyal, Guolin Ke, Chien-Feng Liao, Mirco Ravanelli, et al. 2021. Transformers with competitive ensembles of independent mechanisms. *arXiv e-prints* (2021), arXiv:2103.00336.
- [62] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A lite BERT for self-supervised learning of language representations. In *ICLR*.
- [63] Yann LeCun, John S. Denker, and Sara A. Solla. 1990. Optimal brain damage. In *NIPS*. 598–605.
- [64] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer normalization. *arXiv e-prints* (2016), arXiv:1607.06450.
- [65] Mohan Li, Catalin Zorila, and Rama Doddipatla. 2020. Transformer-based online speech recognition with decoder-end adaptive computation steps. *arXiv e-prints* (2020), arXiv:2011.13834.
- [66] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyu Zhou, Wenhui Chen, Yu-Xiang Wang, et al. 2019. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. In *NeurIPS*. 5244–5254.

- [67] Zhiyuan Li, Kaifeng Lyu, and Sanjeev Arora. 2020. Reconciling modern deep learning with traditional optimization analyses: The intrinsic learning rate. In *NeurIPS*, Vol. 33, 14544–14555.
- [68] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2021. A Survey of Transformers. arxiv:2106.04554.
- [69] Chunxi Liu, Frank Zhang, Duc Le, Suyoun Kim, Yatharth Saraf, and Geoffrey Zweig. 2021. Improving RNN transducer based ASR with auxiliary tasks. In *SLT*. 172–179.
- [70] Hanxiao Liu, Zihang Dai, David R. So, and Quoc V. Le. 2021. Pay attention to MLPs. *ArXiv abs/2105.08050* (2021).
- [71] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. DARTS: Differentiable architecture search. In *ICLR*.
- [72] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, et al. 2020. On the variance of the adaptive learning rate and beyond. In *ICLR*.
- [73] Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. 2020. Understanding the difficulty of training transformers. In *EMNLP*. 5747–5763.
- [74] Yinhan Liu, Mylène Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, et al. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *CoRR abs/1907.11692* (2019).
- [75] Alexandra Luccioni, Alexandre Lacoste, and Victor Schmidt. 2020. Estimating carbon emissions of artificial intelligence [opinion]. *IEEE Technol. Societ. Mag.* 39, 2 (2020), 48–51.
- [76] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. 2017. The concrete distribution: A continuous relaxation of discrete random variables. In *ICLR*.
- [77] Matt Mahoney. 2011. Large Text Compression Benchmark. Retrieved from <http://matmahoney.net/dc/text.html>.
- [78] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture models. In *ICLR*.
- [79] Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In *NeurIPS*, Vol. 32.
- [80] Paulius Mikićevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, et al. 2018. Mixed precision training. In *ICLR*.
- [81] Nikita Nangia and Samuel R. Bowman. 2018. ListOps: A diagnostic dataset for latent tree learning. In *NAACL*.
- [82] Sharan Narang, Hyung Won Chung, Yi Tay, William Fedus, Thibault Fevry, Michael Matena, et al. 2021. Do transformer modifications transfer across implementations and applications? *arXiv e-prints* (2021), arXiv:2102.11972.
- [83] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization. In *EMNLP*. 1797–1807.
- [84] OpenAI. 2013. Saving memory using gradient-checkpointing. Retrieved from <https://github.com/openai/gradient-checkpointing>.
- [85] Mylène Ott, Sergey Edunov, David Grangier, and Michael Auli. 2018. Scaling neural machine translation. In *ML*. 1–9.
- [86] Daniel S. Park, Yu Zhang, Chung-Cheng Chiu, Youzheng Chen, Bo Li, William Chan, et al. 2020. SpecAugment on large scale datasets. In *ICASSP*. 6879–6883.
- [87] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*. 8024–8035.
- [88] Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. Efficient neural architecture search via parameter sharing. In *ICML*, Vol. 80, 4092–4101.
- [89] Sai Prasanna, Anna Rogers, and Anna Rumshisky. 2020. When BERT plays the lottery, all tickets are winning. In *EMNLP*. 3208–3229.
- [90] Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. 2020. Blockwise self-attention for long document understanding. In *EMNLP*. 2555–2565.
- [91] Alec Radford and Karthik Narasimhan. 2018. Improving language understanding by generative pre-training. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [92] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. (2019). https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [93] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. 2020. Compressive transformers for long-range sequence modelling. In *ICLR*.
- [94] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *EMNLP*. 2383–2392.
- [95] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized evolution for image classifier architecture search. In *AAAI*. 4780–4789.
- [96] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Trans. Assoc. Computat. Ling.* 9 (2021), 53–68.
- [97] D. E. Rumelhart, P. Smolensky, J. L. McClelland, and G. E. Hinton. 1986. Schemata and sequential thought processes in PDP models. *Parallel Distributed Processing: Explorations in the Microstructure, Vol. 2: Psychological and Biological Models*, MIT Press, Cambridge, MA, 7–57.

- [98] Hassan Sajjad, Fahim Dalvi, Nadir Durrani, and Preslav Nakov. 2020. On the effect of dropping layers of pre-trained transformer models. *arXiv e-prints* (2020), arXiv:2004.03844.
- [99] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *CoRR abs/1910.01108* (2019).
- [100] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. 2018. How does batch normalization help optimization? In *NeurIPS*, Vol. 31.
- [101] Sofia Serrano and Noah A. Smith. 2019. Is attention interpretable? In *ACL*. 2931–2951.
- [102] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. Self-attention with relative position representations. In *NAACL-HLT*. 464–468.
- [103] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, et al. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*.
- [104] Han Shi, Jiahui Gao, Xiaozhe Ren, Hang Xu, Xiaodan Liang, Zhenguo Li, et al. 2021. SparseBERT: Rethinking the importance analysis in self-attention. In *ICML*, Vol. 139, 9547–9557.
- [105] David R. So, Quoc V. Le, and Chen Liang. 2019. The evolved transformer. In *ICML*, Vol. 97, 5877–5886.
- [106] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* 15, 1 (2014), 1929–1958.
- [107] Pierre Stock, Angela Fan, Benjamin Graham, Edouard Grave, Rémi Gribonval, Herve Jegou, et al. 2021. Training with quantization noise for extreme model compression. In *ICLR*.
- [108] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in NLP. *CoRR abs/1906.02243* (2019).
- [109] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and policy considerations for modern deep learning research. *Proc. AAAI Conf. Artif. Intell.* 34, 09 (2020), 13693–13696.
- [110] Cem Subakan, Mirco Ravanelli, Samuele Cornell, Mirko Bronzi, and Jianyuan Zhong. 2021. Attention is all you need in speech separation. In *ICASSP*. 21–25.
- [111] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. Adaptive attention span in transformers. In *ACL*. 331–335.
- [112] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *NIPS*. 3104–3112.
- [113] Yi Tay, Dara Bahri, Donald Metzler, Da-Cheng Juan, Zhe Zhao, and Che Zheng. 2020. Synthesizer: Rethinking self-attention in transformer models. *arXiv e-prints* (2020), arXiv:2005.00743.
- [114] Yi Tay, Dara Bahri, Liu Yang, Donald Metzler, and Da-Cheng Juan. 2020. Sparse sinkhorn attention. In *ICML*, Vol. 119, 9438–9447.
- [115] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, et al. 2021. Long range arena : A benchmark for efficient transformers. In *ICLR*.
- [116] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2020. Efficient transformers: A survey. *CoRR abs/2009.06732* (2020).
- [117] Wilson L. Taylor. 1953. “Cloze Procedure”: A new tool for measuring readability. *Journal. Quart.* 30, 4 (1953), 415–433.
- [118] Ilya O. Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, et al. 2021. MLP-Mixer: An all-MLP architecture for vision. *CoRR abs/2105.01601* (2021).
- [119] Henry Tsai, Jayden Ooi, Chun-Sung Ferng, Hyung Won Chung, and Jason Riesa. 2020. Finding fast transformers: One-shot neural architecture search by component composition. *arXiv e-prints* (2020), arXiv:2008.06808.
- [120] Henry Tsai, Jason Riesa, Melvin Johnson, Naveen Arivazhagan, Xin Li, and Amelia Archer. 2019. Small and practical BERT models for sequence labeling. In *EMNLP-IJCNLP*. 3632–3636.
- [121] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, et al. 2017. Attention is all you need. In *NIPS*. 5998–6008.
- [122] Apoorv Vyas, Angelos Katharopoulos, and François Fleuret. 2020. Fast transformers with clustered attention. In *NeurIPS*.
- [123] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, et al. 2019. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *NeurIPS*, Vol. 32.
- [124] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *EMNLP*. 353–355.
- [125] Chenguang Wang, Zihao Ye, Aston Zhang, Zheng Zhang, and Alexander J. Smola. 2020. Transformer on a diet. *arXiv e-prints* (2020), arXiv:2002.06170.
- [126] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv e-prints* (2020), arXiv:2006.04768.

- [127] Yuxin Wang, Qiang Wang, Shaohuai Shi, Xin He, Zhenheng Tang, Kaiyong Zhao, et al. 2020. Benchmarking the performance and energy efficiency of AI accelerators for AI training. In *CCGRID*. 744–751.
- [128] Yu Wang, Gu-Yeon Wei, and David Brooks. 2019. Benchmarking TPU, GPU, and CPU platforms for deep learning. *CoRR* abs/1907.10701. <http://arxiv.org/abs/1907.10701>
- [129] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. 2019. Neural network acceptability judgments. *Trans. Assoc. Computat. Ling.* 7 (2019), 625–641.
- [130] Lilian Weng. 2018. Attention? Attention! Retrieved from <http://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html>.
- [131] Sarah Wiegrefe and Yuval Pinter. 2019. Attention is not not explanation. In *EMNLP-IJCNLP*. 11–20.
- [132] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. 2020. Transformers: State-of-the-art natural language processing. In *EMNLP*. 38–45.
- [133] Zhanghao Wu, Zhijian Liu, Ji Lin, Yujun Lin, and Song Han. 2020. Lite transformer with long-short range attention. In *ICLR*.
- [134] Qizhe Xie, Minh-Thang Luong, Eduard H. Hovy, and Quoc V. Le. 2020. Self-training with noisy student improves ImageNet classification. In *CVPR*. 10684–10695.
- [135] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, et al. 2020. On layer normalization in the transformer architecture. In *ICML*, Vol. 119, 10524–10533.
- [136] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, et al. 2021. Nyström-former: A Nyström-based algorithm for approximating self-attention. *arXiv e-prints* (2021), arXiv:2102.03902.
- [137] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. 2022. MetaFormer is actually what you need for vision. In *CVPR*. 10819–10829.
- [138] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8BERT: Quantized 8Bit BERT. In *EMC2-NIPS*.
- [139] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, et al. 2020. Big bird: Transformers for longer sequences. In *NeurIPS*, Vol. 33, 17283–17297.
- [140] Qian Zhang, Han Lu, Hasim Sak, Anshuman Tripathi, Erik McDermott, Stephen Koo, et al. 2020. Transformer transducer: A streamable speech recognition model with transformer encoders and RNN-T loss. In *ICASSP*. 7829–7833.
- [141] Barret Zoph and Quoc V. Le. 2017. Neural architecture search with reinforcement learning. In *ICLR*.
- [142] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V. Le. 2018. Learning transferable architectures for scalable image recognition. In *CVPR*. 8697–8710.

Received 27 January 2022; revised 5 October 2022; accepted 23 February 2023